TUM School of Computation, Information and Technology
Technical University of Munich

Master's Thesis in Robotics, Cognition, Intelligence

Learning Long-Horizon Cyclic Nut Threading with a Dexterous Hand via Hybrid-Teacher Policy Distillation

Lernen langfristiger zyklischer Mutter-Gewindemontage mit einer geschickten Hand durch Policy-Destillation eines hybriden Lehrers

Supervisor Prof. Dr.-Ing. habil. Alois C. Knoll

Advisor Chi Zhang, M.Sc.

Author Carsten Oertel

Date November 15, 2026 in Munich

Disclaimer

I confirm that this Master's Thesis is my own work and I have documented all sources and material used.

Munich, November 15, 2026

(Carsten Oertel)

Acknowledgments

This thesis was conducted at the Chair of Robotics, Artificial Intelligence and Real-time Systems of the Technical University of Munich in collaboration with Agile Robots SE, Munich. The author gratefully acknowledges the supervision of Prof. Dr.-Ing. habil. Alois C. Knoll, the advising of Chi Zhang, M.Sc., whose work on the hybrid teacher, the distillation stack, and the hardware experiments this thesis builds on, and the industrial supervision of Antonio Gonzales Marin, M.Sc. (Agile Robots SE).

Abstract

Threading a nut onto a bolt with a multi-fingered hand is a long-horizon, contact-rich assembly task. A single turn is limited by the range of the wrist, so the hand must grasp, turn, release, return, and re-grasp the nut in repeated cycles, and small errors at the boundaries between these phases accumulate over the cycles. This thesis studies how such a cyclic skill can be learned for a Franka Research 3 arm with an Inspire RH56DFX dexterous hand and transferred from simulation to hardware. Learning all phases end to end with one reinforcement-learning policy did not succeed, because a reward that captures both long-horizon progress and precise phase transitions could not be designed. The thesis therefore follows the principle of decomposing during data generation and unifying at deployment. A hybrid teacher combines a privileged reinforcement-learning policy for approach, grasp, rotation, and goal reaching with scripted release-and-return controllers to generate complete multi-cycle demonstrations in a simplified thread simulation. Replaying these demonstrations on the physical robot with sparse human corrections yields a mixed dataset that reduces the sim-to-real gap. The demonstrations are distilled into a single visuomotor student by conditional flow matching: a trajectory Transformer predicts eight-step action chunks from point-cloud features of an aligned RGB-D observation, proprioception, a global trajectory-progress scalar, and a process-phase encoding. The thesis derives the flow-matching objective, describes the student as implemented, and compares offline flow matching with online DAgger variants, deterministic LSTM and Transformer regression heads, point-cloud and image encoders, privileged and visual observations, and two action contracts. In ten-cycle trials the distilled policy completes all cycles on M24, M30, and M36 hexagonal nuts in simulation and on hardware, with mean cyclic tracking errors below 6°, and transfers to unseen round and octagonal profiles, while rectangular profiles expose a limit of the learned re-grasp strategy.

Kurzfassung

Das Aufschrauben einer Mutter auf einen Bolzen mit einer mehrfingrigen Hand ist eine langfristige, kontaktreiche Montageaufgabe. Eine einzelne Drehung ist durch den Bewegungsbereich des Handgelenks begrenzt, sodass die Hand die Mutter in wiederholten Zyklen greifen, drehen, loslassen, zurückkehren und erneut greifen muss; kleine Fehler an den Phasengrenzen akkumulieren über die Zyklen. Diese Arbeit untersucht, wie eine solche zyklische Fertigkeit für einen Franka-Research-3-Arm mit einer geschickten Inspire-RH56DFX-Hand gelernt und von der Simulation auf die Hardware übertragen werden kann. Das Ende-zu-Ende-Lernen aller Phasen mit einer einzigen Reinforcement-Learning-Policy gelang nicht, da keine Belohnungsfunktion gefunden wurde, die sowohl langfristigen Fortschritt als auch präzise Phasenübergänge erfasst. Die Arbeit folgt daher dem Prinzip, bei der Datenerzeugung zu zerlegen und beim Einsatz zu vereinen. Ein hybrider Lehrer kombiniert eine priv-

ilegierte Reinforcement-Learning-Policy für Annäherung, Griff, Drehung und Zielerreichung mit skriptgesteuerten Loslass- und Rückkehrreglern und erzeugt so vollständige mehrzyklische Demonstrationen in einer vereinfachten Gewindesimulation. Die Wiedergabe dieser Demonstrationen auf dem realen Roboter mit sparsamen menschlichen Korrekturen liefert einen gemischten Datensatz, der die Lücke zwischen Simulation und Realität verringert. Die Demonstrationen werden mittels bedingtem Flow Matching in eine einzige visuomotorische Schüler-Policy destilliert: Ein Trajektorien-Transformer sagt Aktionsblöcke von acht Schritten aus Punktwolkenmerkmalen einer ausgerichteten RGB-D-Beobachtung, Propriozeption, einem globalen Fortschrittsskalar und einer Prozessphasenkodierung vorher. Die Arbeit leitet das Flow-Matching-Ziel her, beschreibt den Schüler wie implementiert und vergleicht offline Flow Matching mit online DAgger-Varianten, deterministischen LSTM- und Transformer-Regressionsköpfen, Punktwolken- und Bildkodierern, privilegierten und visuellen Beobachtungen sowie zwei Aktionsschnittstellen. In Versuchen mit zehn Zyklen vollendet die destillierte Policy alle Zyklen auf M24-, M30- und M36-Sechskantmuttern in Simulation und auf der Hardware mit mittleren zyklischen Nachführfehlern unter 6° und überträgt sich auf ungesehene runde und achteckige Profile, während rechteckige Profile eine Grenze der gelernten Nachgreifstrategie aufzeigen.

Contents

List of Abbreviations **xiii**

List of Symbols **xv**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Setup . . . 2
1.3 Research Questions . . . 2
1.4 Contributions . . . 3
1.5 Outline . . . 3

2 State of the Art **5**
2.1 Theoretical Background . . . 5
2.1.1 Reinforcement Learning with Privileged Information . . . 5
2.1.2 Imitation Learning and Compounding Error . . . 6
2.1.3 Generative Action Heads: Diffusion and Flow Matching . . . 7
2.1.4 Visual Representations for Manipulation . . . 7
2.1.5 Operational-Space Control . . . 8
2.2 Related Work . . . 8
2.2.1 Robot Assembly with Dexterous Manipulation . . . 8
2.2.2 Policy Chaining in Long-Horizon Tasks . . . 9
2.2.3 Sim-to-Real Adaptation with Human Intervention . . . 9
2.2.4 Visuomotor Policy Distillation . . . 9

3 Methodology **11**
3.1 Problem Formulation . . . 11
3.2 Simulated and Physical System . . . 12
3.2.1 Platform . . . 12
3.2.2 Simplified Thread Simulation . . . 13
3.2.3 Action Contract and Controller . . . 14
3.3 Privileged Teacher Training . . . 15
3.4 Hybrid Teacher and Demonstration Generation . . . 17
3.4.1 Cyclic Teacher Rollout in Simulation . . . 17
3.4.2 Full-Task Variant with Pickup and Handover . . . 18
3.4.3 Disturbance Augmentation . . . 18
3.4.4 Human-Corrected Real-World Replay . . . 19
3.4.5 Dataset Format . . . 19
3.5 Student Policy Distillation by Conditional Flow Matching . . . 19
3.5.1 From Behavior Cloning to Online Distillation for Cyclic Tasks . . . 19
3.5.2 Flow Matching for Conditional Action Chunks . . . 20
3.5.3 Student Architecture as Implemented . . . 21

3.5.4 Offline Training . . . 23
3.5.5 Online DAgger with a Flow-Matching Head . . . 23
3.5.6 Baseline Students . . . 25
3.6 Evaluation Protocol . . . 25
3.6.1 Evaluation Ladder . . . 25
3.6.2 Metrics . . . 26

4 Experiments in Simulation 27
4.1 Experimental Setup . . . 27
4.2 Teacher Training . . . 27
4.2.1 End-to-End Threading Does Not Train . . . 27
4.2.2 Threading and Pickup Experts . . . 28
4.3 Student Ablations in Simulation . . . 29
4.4 Design Ablations of the Student . . . 31
4.4.1 Action Contract . . . 31
4.4.2 Progress and Phase Conditioning . . . 31
4.4.3 Vision Encoder and Pretraining . . . 32
4.4.4 Privileged Observations, Horizon, Integration Steps, and Inputs . . . 33
4.4.5 Nut-Size Conditioning, Domain Randomization, and Force Observations 33
4.5 Online Distillation Baselines . . . 34
4.6 Ablation Inventory . . . 35

5 Experiments on Real Hardware 37
5.1 Hardware Setup . . . 37
5.2 Human-Corrected Replay . . . 39
5.3 Results on Training Nut Sizes . . . 39
5.4 Generalization across Nut Sizes and Geometries . . . 40
5.5 Long-Horizon Stress . . . 41

6 Discussion 43
6.1 Interpretation of the Ablation Study . . . 43
6.2 Offline versus Online Distillation . . . 44
6.3 Action and Observation Contracts . . . 45
6.4 Answers to the Questions of the Exposé . . . 45
6.5 Limitations . . . 46

7 Conclusion and Future Work 47
7.1 Conclusion . . . 47
7.2 Future Work . . . 47

A Hyperparameters and Interfaces 49
A.1 Teacher Observations . . . 49
A.2 Teacher Hyperparameters . . . 49
A.3 Student Hyperparameters . . . 49
A.4 Dataset Summary . . . 49

Bibliography 53

List of Figures

3.1 Overview of the pipeline. Decompose during data generation, unify at deployment: a privileged RL policy executes approach, grasp, rotation, and goal reaching, scripted controllers execute release and return, and together they form the hybrid teacher that generates multi-cycle demonstrations in simulation. Replaying the demonstrations on the robot with human corrections yields real-world demonstrations. Conditional flow matching distills the combined dataset into one visuomotor student that executes the complete cyclic task from RGB-D and proprioception. 12

3.2 Simplified thread simulation. A passive revolute joint θ_t about the bolt axis is kinematically coupled to a position-controlled prismatic joint d_t with $d_t = k\theta_t$ and $k = -\rho/(2\pi)$, so that the nut advances by one pitch per revolution without explicit thread contacts. Figure reproduced from the accompanying paper [**NUTS2026**]. 14

3.3 Data-collection loop of the hybrid teacher for one threading cycle. The learned policy owns approach, grasp, turn, and hold; a physical release gate hands over to the scripted release, four retreat waypoints, and the gated return; a completed return rebases the turning target and starts the next cycle. Every executed command is recorded in the common 9-D label space. 18

3.4 Student architecture. The aligned RGB-D observation is back-projected into a point cloud and encoded into a 64-D visual feature; the 29-D proprioception is encoded by an MLP; late fusion concatenates the two with the visual feature first and projects to the 256-D condition, to which learned embeddings of the global trajectory progress and of the process phase are added. A non-causal trajectory Transformer predicts the flow velocity of all eight action tokens given the condition token and a sinusoidal embedding of the flow time. An auxiliary head regresses the nut position from the visual feature during training. 21

3.5 Evaluation ladder. Offline teacher-fit feeds recorded teacher observations to the student and compares predicted chunks with recorded commands without a simulator. Teacher-forced clock executes the student's commands in live physics while the student receives the recorded teacher observations on the recorded timeline, removing observation drift. Normal closed loop feeds live observations back to the student and is the only mode that measures compounding error. 25

4.1 Simulation scenes. Teacher policies are trained in hundreds of parallel workcells; demonstrations for the student are rendered with randomized dome lighting, table, object, floor, and background textures, from which the student sees aligned RGB and depth. Images reproduced from the accompanying paper [**NUTS2026**]. 28

5.1 Real-world setup: FR3 arm with the Inspire hand, the bolt on its pedestal, and the fixed RealSense D415. Images reproduced from the accompanying paper [**NUTS2026**]. 38
5.2 Nut-angle measurement. An angular scale under the bolt is read at the end of every goal-reaching phase to compute the cyclic and ten-cycle tracking errors; from left to right, grasp, turn, and release of one cycle. Images reproduced from the accompanying paper [**NUTS2026**]. 39

List of Tables

3.1 Domain randomization during teacher training. $U(a, b)$ denotes uniform sampling; multiplicative factors are sampled as $1 + U(0, \ell)$ or its reciprocal with equal probability. 16

4.1 Teacher training results. Success is the population success rate at the end of training; the hybrid-teacher column counts candidate environments that completed the required cycles with zero transition failures. 29
4.2 Simulation ablations on the M24 hexagonal nut over ten trials. Each defined entry reports mean ± standard deviation. Completed cycles are averaged across trials, cyclic-error statistics use completed cycles only, and the ten-cycle error compares the accumulated rotation with the 600° target; it is reported as "–" when no trial completes all ten cycles. Errors are signed in degrees: negative denotes lag and positive overshoot. Results from the accompanying paper [**NUTS2026**]. 30
4.3 Action-contract ablation of the offline flow student on pickup demonstrations (privileged 6-D observation, single seed). G0 is the handoff waypoint above the bolt tip and G1 the engagement goal. 31
4.4 Conditioning ablation on six-cycle sequential threading (privileged 6-D observation, matched single rollouts at one seed). RMSE values are against the nominal teacher trajectory in absolute time; the final turn error is the cumulative nut-rotation error after the last cycle. 32
4.5 Vision-encoder ablation of the flow student on the ten-cycle M24 task (ten trials). The DP3 row is the full method; the remaining rows are placeholders. 33
4.6 Student hyperparameter sweeps on the ten-cycle M24 task (ten trials). All cells are placeholders except the reference row. 33
4.7 Robustness ablations in simulation (ten trials of ten cycles). All cells are placeholders. 34
4.8 Online distillation baselines on cyclic threading (privileged 6-D observation, Cartesian contract). Cycles are student-only completed cycles of the best retained checkpoint in one evaluation rollout; the offline rows are matched single rollouts from Section 4.4. 34
4.9 Offline teacher-fit against closed-loop outcome for offline flow students (EMA weights, sixteen Euler steps, one training episode). RMSE values are in the student's action units; the joint-PD rows are in physical units, the Cartesian rows in normalized action units. 35
4.10 Inventory of ablations supported by the code base. "Result" names the table or section with a result; TBD marks an ablation whose result is pending. 35

5.1 Real-world ablations on the M24 hexagonal nut over ten trials, with the same conventions as Table 4.2. Ablations (c) and (d) were not tested on hardware. Results from the accompanying paper [**NUTS2026**]. 40

5.2 Geometry-dependent real-world transfer across nut sizes and profiles over ten trials per nut. Values are mean ± standard deviation; the octagonal, rectangular, and round rows are out-of-distribution geometries. Results from the accompanying paper [**NUTS2026**]. 40

A.1 Actor and critic observations of the single-size M24 threading teacher. Noise is applied to the actor only. The multi-size teacher appends a clean 3-D one-hot nut-size code to both. 49
A.2 PPO hyperparameters of the privileged teacher (rl_games, asymmetric actor–critic). 50
A.3 Hyperparameters of the conditional flow-matching student. 50
A.4 Demonstration data used for the reported students. 51

List of Abbreviations

ACT	Action Chunking with Transformers
ADR	Automatic Domain Randomization
BC	Behavior Cloning
CFM	Conditional Flow Matching
DAgger	Dataset Aggregation
DoF	Degree of Freedom
DP3	3D Diffusion Policy
DR	Domain Randomization
EMA	Exponential Moving Average
FR3	Franka Research 3
IL	Imitation Learning
LSTM	Long Short-Term Memory
MDP	Markov Decision Process
MLP	Multilayer Perceptron
ODE	Ordinary Differential Equation
OSC	Operational-Space Control
PD	Proportional-Derivative
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
RGB-D	Color and Depth Image
RL	Reinforcement Learning
RMSE	Root Mean Square Error
SDF	Signed Distance Field
TCP	Tool Center Point

List of Symbols

The following symbols are used with the same meaning in all chapters.

Symbol	Meaning
Task	
$\mathcal{S}$	ordered phase sequence of one threading cycle
$m_t \in \mathcal{M} = \mathcal{M}_{\mathrm{RL}} \cup \mathcal{M}_{\mathrm{script}}$	active phase at step t; learned phases $\mathcal{M}_{\mathrm{RL}}$, scripted phases $\mathcal{M}_{\mathrm{script}}$
K	number of cycles per trial ($K = 10$)
$\Delta\varphi^*$	desired clockwise nut rotation per cycle (60°)
φ_k	unwrapped nut angle after the goal-reaching phase of cycle k
e_k, e_{ep}	cyclic and episodic (accumulated) tracking error
$C(\tau)$, $E(\tau)$	completed cycles and tracking cost of a trajectory τ
ρ	thread pitch
θ_t, d_t	revolute and prismatic coordinate of the simplified thread joint
Teacher	
$\boldsymbol{x}_t$	full simulator state
$\boldsymbol{z}_t^{\pi} = g_{\pi}(\boldsymbol{x}_t, \boldsymbol{\xi}_t)$	actor observation with observation noise $\boldsymbol{\xi}_t$
$\boldsymbol{h}_t$	recurrent hidden state of the actor
$\pi_{\mathrm{T}}(\boldsymbol{a} \mid \boldsymbol{z}^{\pi}, \boldsymbol{h})$, $\bar{\pi}_{\mathrm{T}}$	privileged RL teacher and its deterministic mean
κ_m	scripted controller of transition phase $m \in \mathcal{M}_{\mathrm{script}}$
π_{HT}, π_{H}	hybrid teacher and human operator
$\boldsymbol{a}_t \in [-1, 1]^9$	normalized Cartesian-controller action
r_t, $R_{i,t}$, $P_{j,t}$	shaped reward, reward terms, penalty terms
Student	
$\boldsymbol{y}_t = (\boldsymbol{O}_t^{\mathrm{RGBD}}, \boldsymbol{s}_t, p_t, \boldsymbol{e}_t^m)$	student observation: RGB-D image, proprioception, progress, phase encoding
$\mathcal{P}_t$, $\boldsymbol{v}_t \in \mathbb{R}^{64}$	point cloud and visual feature
$\boldsymbol{c}_t \in \mathbb{R}^{256}$	fused condition vector
H	action horizon (chunk length, $H = 8$)
$\boldsymbol{U}_t^D = [\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}]$	demonstrated action chunk
$\boldsymbol{U}_t^0 \sim \mathcal{N}(\boldsymbol{0}, I)$	Gaussian source sample
$\lambda \in [0, 1]$	flow time
$\boldsymbol{U}_t^{\lambda}$	interpolant between source and target chunk
$\boldsymbol{v}_{\theta}(\boldsymbol{U}^{\lambda}, \lambda \mid \boldsymbol{c})$	learned conditional velocity field
N	number of ODE integration steps
β	probability of teacher control during online DAgger
$\mathcal{D}_{\mathrm{sim}}$, $\mathcal{D}_{\mathrm{real}}$, $\mathcal{D}$	simulated, real-world, and combined demonstration sets
Control	
$\boldsymbol{q}$, $\dot{\boldsymbol{q}}$	joint positions and velocities
$\boldsymbol{\tau}$	joint torques
J, M, Λ	geometric Jacobian, joint-space inertia, task-space inertia
K_p, K_d	proportional and derivative gains

Chapter 1

Introduction

This chapter motivates learning nut threading with a dexterous hand, defines the cyclic threading problem addressed in this thesis, states the research questions and contributions, and outlines the remaining chapters.

1.1 Motivation

Endowing robots with human-like dexterity is a long-standing goal of robotics, and one of its most convincing demonstrations is the ability to handle the parts and tools of a world built for human hands [BK00]. A multi-fingered hand promises to pick up objects of varying shape, hold and reorient them, and operate the very implements that people use every day. Learning-based methods have begun to realize this promise, and in-hand manipulation in particular has advanced rapidly in recent years [Wei+24; CXA21; Han+24]. Among the most demanding of these skills are those of mechanical assembly, where a firm grasp of a part must be coupled to a fine, contact-rich motion that joins it to another part.

Threading a nut onto a bolt is one of the most common operations in industrial assembly and, at the same time, one of the hardest to automate. The parts mate through sub-millimeter thread geometry, progress is governed by friction and contact forces that are difficult to sense, and success couples a stable grasp of the fastener to a controlled turning motion of the grasped part. Reinforcement learning (RL) in high-fidelity GPU simulation has recently made contact-rich assembly learnable: Factory [Nar+22] simulates nut-and-bolt engagement with thousands of simultaneous contacts in real time, and FORGE [Nos+25] and IndustReal [Tan+23] transfer such policies to physical robots. These systems, however, act with a parallel-jaw gripper and begin each episode with a fastener that is already held. The policy only learns to turn a pre-grasped part; grasp acquisition, grasp maintenance under turning, and the re-grasping that a full thread run-down requires are left outside the learning problem.

A dexterous hand changes the nature of the task. Humans thread nuts in several ways: a single finger suffices for free spinning, a pinch grasp is preferred for controlled threading, and a tripod or power grasp is applied under high resistance [Fei+16]. With a pinch grasp, one turn is bounded by the range of the wrist and the fingers, so a nut has to be turned, released, re-approached, and grasped again, cycle after cycle, while the nut advances along the bolt by one pitch per revolution. The task is therefore long-horizon and cyclic. Its difficulty does not lie in any single phase but in the transitions between phases: a grasp that is slightly off the intended faces, a release that disturbs the nut, or a return that lands the fingers at the wrong height propagates into the next cycle and compounds over ten or more cycles. Prior learning-based work on dexterous threading has addressed the spinning motion of an already engaged

fastener [**Hsieh2026**], and goal-conditioned screwing of furniture parts [**Lum2026**], but not controlled, goal-directed, multi-cycle threading of everyday nuts with learned grasp and re-grasp.

1.2 Problem Setup

This thesis studies the task of threading a hexagonal nut onto a vertically mounted bolt with a seven-degree-of-freedom Franka Research 3 (FR3) arm equipped with an Inspire RH56DFX five-fingered hand. The nut starts engaged on the bolt by a fraction of a turn. One threading cycle consists of approaching the nut, establishing a pinch grasp between thumb and index finger on two opposing faces, rotating the nut clockwise by a target angle of 60° while it advances along the thread, holding the target, releasing the nut, and returning the hand to a re-grasp pose. A trial requires ten such cycles in sequence, and the deployed policy must execute them from an aligned RGB-D camera and the robot's proprioception alone, without privileged object state, scripted controllers, or human assistance.

The skill is learned in simulation and transferred to the physical robot. Simulating threads faithfully is computationally expensive, because engaged threads produce thousands of simultaneous contacts [Nar+22]; the reinforcement-learning stage of this work therefore uses a simplified thread model that couples a passive revolute joint to a position-controlled prismatic joint while retaining the finger–nut contact forces. The simplification creates a sim-to-real gap in contact and perception that the transfer method must close.

Two properties of the task shape the method. First, the task is long-horizon and cyclic: a policy must sustain consecutive cycles and is sensitive to errors at phase boundaries. Second, the task is only partially observable at deployment: the nut pose that the simulator exposes to a privileged policy must be inferred from camera images by the deployed policy. The first property favors generating the training signal phase by phase, where each phase is easy to specify; the second favors a teacher–student scheme in which a privileged teacher is distilled into a sensor-based student [Sin+25; Oer26].

1.3 Research Questions

The exposé of this thesis set out two research questions. The first asked whether a force-aware policy that threads a nut onto a bolt with a dexterous hand–arm system can be learned with proximal policy optimization (PPO) in a privileged simulation without constraining the thread interaction. The second asked whether such a privileged teacher can be distilled into a vision-based student that estimates the object pose with the accuracy required for sub-millimeter thread interaction. Both questions were answered, but not in the form they were posed. Repeated attempts to learn all phases of the cyclic task end to end with a single RL policy on faithfully simulated threads did not produce a threading teacher, because a reward that captures both long-horizon progress and precise transitions between phases could not be designed, and because the unconstrained thread contact admitted exploits such as pressing the nut through the thread. The thesis therefore refines the questions into the four that organize its experiments:

1. Does a single student trained on complete multi-cycle demonstrations outperform repeated execution of a single-cycle policy (Q1)?

2. Which visual representation preserves the geometry needed for repeated re-grasping

under viewpoint and axial change, a point-cloud encoder or an image-space encoder (Q2)?

3. Does a generative flow-matching action head distill the heterogeneous demonstrations more effectively than deterministic regression with recurrent or Transformer backbones (Q3)?

4. Does augmenting simulated demonstrations with sparse human corrections collected during real-world replay close the sim-to-real gap (Q4)?

1.4 Contributions

The main contributions of this thesis are the following.

1. A formulation of long-horizon cyclic nut threading as an ordered sequence of phases with explicit cyclic and episodic tracking errors, and a hybrid teacher that combines a privileged RL policy for approach, grasp, rotation, and goal reaching with scripted release-and-return controllers. The hybrid teacher generates coherent multi-cycle demonstrations without a monolithic RL teacher.

2. A human-corrected sim-to-real demonstration strategy in which simulated demonstrations are replayed on the physical robot and failure-prone segments between grasp and release are selectively overwritten by an operator, so that the demonstration distribution adapts to hardware contact and perception discrepancies.

3. A unified visuomotor student distilled by conditional flow matching from point-cloud features of an aligned RGB-D view, proprioception, a global trajectory-progress scalar, and a process-phase encoding. The thesis derives the objective, documents the student as implemented, and places it against online DAgger variants, deterministic regression heads, alternative vision encoders, privileged observations, and two action contracts in a systematic ablation study.

4. An experimental evaluation in simulation and on hardware. In ten-cycle trials the student completes every trial on the M24, M30, and M36 hexagonal nuts it was trained on and on unseen round and octagonal profiles, and the evaluation exposes rectangular profiles as a failure case of the learned re-grasp strategy.

The method and its main results were submitted as a conference paper [**NUTS2026**]. This thesis expands the paper into a full report: it adds the background on imitation learning and flow matching, the design rationale for the action and observation contracts, the ablations implemented in the accompanying code base, the negative results that motivated the final design, and a discussion of the limitations.

1.5 Outline

Chapter 2 reviews the theoretical background on reinforcement learning, imitation learning, generative action heads, visual representations, and operational-space control, and surveys related work on robotic assembly, policy chaining, sim-to-real transfer, and visuomotor distillation. Chapter 3 presents the methodology: the problem formulation and metrics, the

simulated and physical system, the privileged teacher, the hybrid teacher and demonstration pipeline, the conditional flow-matching student, and the evaluation protocol. Chapter 4 reports the experiments in simulation, including the ablations and the negative results that shaped the design. Chapter 5 describes the hardware setup, the human-corrected replay, and the real-world results across nut sizes and geometries. Chapter 6 interprets the results with respect to the research questions and discusses the limitations. Chapter 7 concludes and outlines future work.

Chapter 2

State of the Art

This chapter has two parts. Section 2.1 introduces the theoretical concepts the thesis builds on: reinforcement learning with privileged critics, imitation learning and its compounding-error problem, interactive data aggregation, generative action heads based on diffusion and flow matching, visual representations for manipulation, and operational-space control. Section 2.2 reviews related work on robotic assembly with dexterous hands, policy chaining for long-horizon tasks, sim-to-real transfer with human intervention, and visuomotor policy distillation.

2.1 Theoretical Background

2.1.1 Reinforcement Learning with Privileged Information

Reinforcement learning formalizes sequential decision making as a Markov decision process (MDP) $(\mathcal{X}, \mathcal{A}, P, r, \gamma)$ with state space $\mathcal{X}$, action space $\mathcal{A}$, transition kernel $P(\boldsymbol{x}_{t+1} \mid \boldsymbol{x}_t, \boldsymbol{a}_t)$, reward $r(\boldsymbol{x}_t, \boldsymbol{a}_t)$, and discount factor $\gamma \in [0, 1)$ [KBP13]. A policy $\pi(\boldsymbol{a} \mid \boldsymbol{x})$ is sought that maximizes the expected discounted return $J(\pi) = \mathbb{E}_\pi[\sum_t \gamma^t r_t]$. For robot manipulation the full state is not observable at deployment; the problem is then a partially observable MDP (POMDP), in which the policy acts on observations $\boldsymbol{z}_t \sim O(\cdot \mid \boldsymbol{x}_t)$ instead of states.

Proximal policy optimization (PPO) [**Schulman2017**] is the on-policy actor–critic algorithm used throughout this thesis. The actor π_θ is updated by maximizing the clipped surrogate objective

$$L^{\mathrm{PPO}}(\theta) = \mathbb{E}_t\Big[\min\big(\rho_t(\theta)\hat{A}_t,\ \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\big)\Big], \qquad \rho_t(\theta) = \frac{\pi_\theta(\boldsymbol{a}_t \mid \boldsymbol{z}_t)}{\pi_{\theta_{\mathrm{old}}}(\boldsymbol{a}_t \mid \boldsymbol{z}_t)}, \quad (2.1)$$

where $\hat{A}_t$ is an advantage estimate computed from the critic's value prediction. The clipping bounds the policy change per update and makes PPO stable on the thousands of parallel environments that GPU simulators provide.

In an asymmetric actor–critic the actor and the critic receive different inputs [**Pinto2017**]. The critic estimates the value function from the complete simulator state, including quantities that no sensor measures, whereas the actor observes only a noisy subset that can later be reproduced on hardware or distilled into a sensor-based policy. Because the critic is discarded after training, the privileged information shapes the gradients without entering the deployed policy. When the actor observation is itself a noisy or delayed function of the state, a recurrent actor with a long short-term memory (LSTM) cell [**Hochreiter1997**] integrates the observation history into a hidden state $\boldsymbol{h}_t$ and thereby recovers part of the information lost to partial

observability. This recipe, together with domain randomization of physical and controller parameters, underlies most recent sim-to-real results in dexterous manipulation [Han+24; Pet+23; Lum+24; Sin+25].

2.1.2 Imitation Learning and Compounding Error

Imitation learning (IL) trains a policy from demonstrations rather than from a reward. Given a dataset $\mathcal{D} = \{(\boldsymbol{y}_t, \boldsymbol{u}_t^*)\}$ of observations and expert actions, behavior cloning (BC) fits a policy by supervised regression,

$$\theta^* = \arg\min_{\theta} \mathbb{E}_{(\boldsymbol{y},\boldsymbol{u}^*)\sim\mathcal{D}}[\ell(\pi_\theta(\boldsymbol{y}), \boldsymbol{u}^*)], \tag{2.2}$$

with a per-sample loss ℓ such as the squared error. BC is simple and data-efficient, but it is trained on the state distribution induced by the expert and evaluated on the state distribution induced by the student. Small prediction errors move the student into states the expert never visited, where its predictions are unreliable, and the error grows with the horizon. Ross et al. [RGB11] show that the expected cost of a BC policy with per-step error ε can grow as $O(T^2\varepsilon)$ over a horizon T, because an error at step t affects all subsequent steps. For a cyclic task of several hundred control steps this quadratic growth is the central obstacle: a policy that reproduces each phase well in isolation can still fail after a few cycles once small deviations in grasp height or nut angle accumulate.

Dataset aggregation (DAgger) [RGB11] addresses the distribution mismatch by querying the expert on the states that the student itself visits. In iteration i the environment is driven by a mixture policy that executes the expert's action with probability β_i and the student's action otherwise, the expert's action is recorded as the label for every visited state, the labeled states are added to the aggregated dataset, and the student is retrained. Under a no-regret learner, the resulting policy has expected cost $O(T\varepsilon)$, linear rather than quadratic in the horizon. The guarantee holds because the aggregated dataset converges to the state distribution of the learned policy itself, so that the supervised error measured on that dataset is the error the policy will actually incur; behavior cloning measures its error on the expert's distribution, where the policy is never evaluated. The price is the $O(T)$ expert queries per iteration and the requirement that the expert be queryable at every visited state. The schedule β_i is typically decayed from one toward zero so that early iterations stay close to the expert's distribution and later iterations expose the student to its own mistakes. SafeDAgger [ZC16] reduces the number of expert queries by invoking the expert only when a learned safety classifier, or a threshold on the disagreement between expert and student actions, predicts that the student is about to fail. The predecessor of this thesis applied DAgger and SafeDAgger to distill a privileged grasping teacher into an RGB student and found that the safety-gated variant produced both the best grasp success and the fewest unsafe episodes [Oer26].

DAgger requires an expert that can be queried at arbitrary student-visited states. A privileged simulation policy satisfies this requirement, a human operator or a replayed demonstration does not. This distinction matters for the present work: in simulation the hybrid teacher can label any state, but on hardware the available supervision consists of replayed trajectories and the operator's corrections, which is an offline dataset. The student must therefore be trainable both from online labels and from a fixed dataset, and the method of Chapter 3 is designed for that.

2.1.3 Generative Action Heads: Diffusion and Flow Matching

A deterministic regression head predicts one action for each observation. Demonstrations of manipulation tasks are, however, often multimodal: two demonstrators, or one teacher and one human corrector, may approach the same nut from different sides, and averaging their actions produces an action that neither of them would take. Generative action heads represent the full conditional distribution $p(\boldsymbol{u} \mid \boldsymbol{y})$ and sample from it at run time. Diffusion Policy [Chi+24] models the distribution of action sequences by iterative denoising: Gaussian noise is added to demonstrated action chunks in a forward process, a network is trained to predict the noise, and actions are generated by reversing the process over tens to hundreds of denoising steps. Diffusion policies handle multimodality and predict action chunks, which improves temporal consistency, but their sampling cost is high for control loops with tight latency budgets.

Flow matching [Lip+23; Lip+24] reaches the same goal with a simpler training target and a cheaper sampler. Instead of a stochastic denoising process it learns a time-dependent velocity field $\boldsymbol{v}_\theta(\boldsymbol{x}, \lambda)$ that transports samples of a simple source distribution p_0, typically a standard Gaussian, to the data distribution p_1 along the ordinary differential equation (ODE) $\dot{\boldsymbol{x}} = \boldsymbol{v}_\theta(\boldsymbol{x}, \lambda)$ for $\lambda \in [0, 1]$. The conditional flow-matching objective regresses the velocity field onto the velocity of a prescribed conditional probability path between a source sample and a data sample. With the linear interpolant $\boldsymbol{x}^\lambda = (1-\lambda)\boldsymbol{x}^0 + \lambda \boldsymbol{x}^1$, whose velocity is $\boldsymbol{x}^1 - \boldsymbol{x}^0$, the objective is a plain regression and the learned ODE can be integrated with a handful of Euler steps. Section 3.5.2 derives this objective and states it for conditional action chunks. Diffusion and flow matching are closely related. The forward noising process of a diffusion model defines a Gaussian probability path between data and noise, and its reverse process can equally be written as a deterministic probability-flow ODE driven by the learned score; flow matching with Gaussian conditional paths contains this diffusion path as a special case, and with the optimal-transport path it replaces the curved diffusion trajectories by straight lines that are cheaper to integrate [Lip+23]. The practical consequences are the absence of a noise schedule, a regression target that is a simple difference of two samples, and samplers with few steps. Flow matching has become the action head of choice for large vision-language-action models such as π_0 [Bla+26], which generate 50-step action chunks with ten integration steps, and it is the action head of the student in this thesis.

2.1.4 Visual Representations for Manipulation

A visuomotor student must extract the nut pose relative to the hand from camera data. Three families of encoders are relevant here. Image-space encoders apply a convolutional backbone such as ResNet-18 [**He2016**] to the RGB image, possibly with depth as an additional channel, and pool the feature map into a vector. They inherit ImageNet pretraining but have to learn metric geometry from appearance and must be made robust to texture and lighting with visual domain randomization [**Tobin2017**]. Point-cloud encoders back-project the depth image through the calibrated camera model into a set of 3D points, optionally attach the pixel color, and encode the set with a small shared point network followed by global max pooling in the spirit of PointNet [**Qi2017**]. 3D Diffusion Policy (DP3) [Ze+24] shows that such a compact point encoder generalizes better across viewpoints and appearance than image encoders for diffusion-based visuomotor policies, because the metric geometry enters the representation directly. Adapt3R [**Wilcox2025**] combines the two: dense 2D features from a frozen pretrained backbone are localized in 3D by the depth map and expressed in the end-effector frame before attention pooling, which makes the representation invariant to the embodiment and robust to domain change. All three families are implemented in the

code base of this thesis, and Section 3.5.3 explains why the DP3-style encoder was retained.

2.1.5 Operational-Space Control

Both the teacher and the student command a Cartesian end-effector target that a task-space impedance controller converts to joint torques. Let $\boldsymbol{q} \in \mathbb{R}^7$ be the arm joint positions, $J(\boldsymbol{q})$ the geometric Jacobian of the controlled frame, and $\boldsymbol{e} \in \mathbb{R}^6$ the position and axis-angle orientation error between target and current pose. The controller applies

$$\boldsymbol{\tau} = J^{\mathrm{T}}\big(K_p \boldsymbol{e} - K_d \dot{\boldsymbol{x}}\big) + N^{\mathrm{T}}\big(K_{p,0}(\boldsymbol{q}_0 - \boldsymbol{q}) - K_{d,0}\dot{\boldsymbol{q}}\big), \tag{2.3}$$

where $\dot{\boldsymbol{x}}$ is the end-effector twist, K_p and $K_d = 2\sqrt{K_p}$ are diagonal stiffness and critical-damping gains, and the second term pulls the redundant seventh degree of freedom toward a rest posture $\boldsymbol{q}_0$ through a nullspace projector N. The projector is dynamically consistent when $N^{\mathrm{T}} = I - J^{\mathrm{T}}\bar{J}^{\mathrm{T}}$ with $\bar{J} = M^{-1}J^{\mathrm{T}}(JM^{-1}J^{\mathrm{T}})^{-1}$ and joint-space inertia M [DOA15; Lab19]. Equation (2.3) is the Factory and FORGE controller [Nar+22; Nos+25], which this thesis inherits. Premultiplying the task-space force by the task-space inertia $\Lambda = (JM^{-1}J^{\mathrm{T}})^{-1}$ yields the full operational-space formulation, in which the closed loop becomes a decoupled unit-mass system in every direction. Impedance control is preferred over stiff position control for a contact-rich task because the compliant law yields on unexpected contact rather than driving through it, and because the policy can probe the impedance by commanding pose offsets rather than torques. The compliance is also what allows the simplified thread model of Section 3.2.2 to tolerate small deviations of the hand from the helical path.

2.2 Related Work

2.2.1 Robot Assembly with Dexterous Manipulation

Robot assembly requires precise motion and closed-loop contact under geometric and dynamic uncertainty, with tasks including peg-in-hole insertion, gear meshing, connector mating, screwdriving, and nut–bolt fastening [**Siciliano2016**]. Classical systems rely on fixtures or specialized tools, whereas multi-fingered hands offer greater versatility but introduce high-dimensional control and complex contact dynamics [BK00; Wei+24]. Learning-based systems include Factory [Nar+22] for scalable contact simulation with signed-distance-field collision detection, contact reduction, and a Gauss–Seidel solver, which simulates one thousand parallel nut–bolt assemblies in real time on one GPU; FORGE [Nos+25] for force-guided contact search under pose uncertainty; IndustReal [Tan+23] for transferring such policies to a physical arm; SimToolReal [Ked+26] for zero-shot dexterous tool use; and Play2Perfect [**Lum2026**] for reusable manipulation priors learned through play. Screwdriving with hand tools has been studied with analytical contact models and physics-informed learning [TJ23; TJX24; Man+23], and finger gaiting under contact force control has been demonstrated for pivoting tasks [XTJ23]. Closest to this thesis, DexScrew [**Hsieh2026**] uses a dexterous hand–arm setup similar to ours and replaces explicit thread geometry with a revolute-joint proxy to learn a rotational finger gait; the learned skill assists teleoperation and supports the collection of real-world demonstrations for tactile behavior cloning. Its policy performs uncontrolled spinning without target conditioning or learned termination, leaving goal-directed, cyclic nut threading unresolved. Play2Perfect [**Lum2026**] builds on the goal-conditioned object-manipulation policy of SimToolReal [Ked+26], pretrains it through play,

and fine-tunes it for precise assembly including screwing a rod-like furniture leg; this enables goal-directed screwing in a structured setting but does not address everyday threaded objects such as nuts and bolts. In contrast to both, the method of this thesis performs controlled, multi-cycle nut threading with coordinated turning and release-and-return behavior across multiple nut sizes.

2.2.2 Policy Chaining in Long-Horizon Tasks

Training a single policy for a long-horizon task is challenging: small control errors can compound over time [RGB11], while sequential fine-tuning on new subtasks can overwrite previously acquired behaviors and cause catastrophic forgetting [**Kirkpatrick2017**]. Common remedies include increasing network capacity, freezing or regularizing task-relevant parameters, and adopting architectures with stronger temporal memory such as recurrent networks or Transformers. An alternative is to decompose the task into reusable skills and chain their policies. The decomposition reduces the burden on each individual policy but introduces a transition problem: the terminal states produced by one skill may fall outside the initial-state distribution of the next, causing failures at skill boundaries. Adversarial skill chaining [**Lee2022**] regularizes terminal states to reduce this mismatch. Sequential Dexterity [Che+23] addresses it with a learned transition-feasibility function that fine-tunes adjacent dexterous policies, supports autonomous switching, and enables recovery. Factory [Nar+22] itself divides nut-and-bolt assembly into pick, place, and screw subpolicies. This thesis takes a different route: it decomposes only during data generation, where each phase has a simple specification, and unifies at deployment by distilling the chained demonstrations into one policy that owns the transitions.

2.2.3 Sim-to-Real Adaptation with Human Intervention

The sim-to-real gap is a long-standing problem in robot learning: policies trained in simulation encounter mismatches in perception, dynamics, contact, and control when deployed on physical hardware. These discrepancies become increasingly consequential for high-dimensional, contact-rich dexterous manipulation. Most approaches seek robust zero-shot transfer through domain randomization [**Tobin2017**; Han+24] or domain adaptation of the visual representation [**Wilcox2025**], while methods such as TRANSIC [**Jiang2025transic**] use limited real-world data, collected as human corrections of a simulation-trained policy, to adapt the policy. Human-in-the-loop reinforcement learning [Luo+25] and imitation-bootstrapped online RL [Hua+25] likewise rely on sparse operator interventions to reach precise manipulation on hardware. DAgger [RGB11] aggregates expert labels on states visited by the learner, whereas SafeDAgger [ZC16] reduces expert queries by requesting supervision only when the learned policy is predicted to be unsafe. Similarly, during real-world replay of the hybrid teacher in this thesis, anticipated slip or misalignment failures are replaced with human corrections and the corrected trajectories are added to the student-distillation dataset.

2.2.4 Visuomotor Policy Distillation

Learning by Cheating [**Chen2020**] demonstrates a practical route from policies trained with privileged simulator state to deployable policies that act from onboard sensing. In dexterous manipulation, DextrAH-G [Lum+24] and DextrAH-RGB [Sin+25] distill a privileged

reinforcement-learning teacher into end-to-end depth- and RGB-based grasping policies with online DAgger, and UniGraspTransformer [**Wang2025**] consolidates object-specific teacher policies into a unified student. The predecessor semester thesis [Oer26] applied this recipe to a 23-DoF hand–arm system and compared BC, DAgger, and SafeDAgger. For long-horizon tasks, predicting action sequences improves temporal consistency: Action Chunking with Transformers (ACT) [**Zhao2023**] predicts chunks of future actions with a Transformer and blends overlapping chunks by temporal ensembling, and generative models can represent multiple valid behaviors. Diffusion Policy [Chi+24] models multimodal action sequences through iterative denoising, DP3 [Ze+24] conditions it on compact point-cloud features, and flow matching [Lip+23] instead learns a continuous vector field for efficient ODE-based action generation, as adopted by π_0 [Bla+26]. These properties motivate its use for distilling the hybrid-teacher and human-corrected demonstrations in this thesis.

Chapter 3

Methodology

This chapter presents the method. Section 3.1 formalizes cyclic nut threading and the metrics that evaluate it. Section 3.2 describes the simulated and physical system: the hand–arm platform, the simplified thread model, and the action contract with its controller. Section 3.3 summarizes the privileged reinforcement-learning teacher. Section 3.4 explains how the hybrid teacher generates complete multi-cycle demonstrations, how they are augmented by disturbances and by human corrections on the physical robot, and how they are stored. Section 3.5 develops the central contribution, the distillation of these demonstrations into a single visuomotor student by conditional flow matching, and relates the offline and online variants. Section 3.6 defines the evaluation protocol. Figure 3.1 gives an overview.

3.1 Problem Formulation

Cyclic nut threading is decomposed into the ordered phase sequence

$$\mathcal{S} = \big(\underbrace{\text{approach, grasp, rotate, goal reaching}}_{\mathcal{M}_{\mathrm{RL}}:\,\pi_{\mathrm{T}}},\ \underbrace{\text{release, return and reset}}_{\mathcal{M}_{\mathrm{script}}:\,\kappa}\big). \tag{3.1}$$

The active phase at control step t is $m_t \in \mathcal{M} = \mathcal{M}_{\mathrm{RL}} \cup \mathcal{M}_{\mathrm{script}}$. The four phases in $\mathcal{M}_{\mathrm{RL}}$ are executed by a learned teacher policy π_{T}; the two transition phases in $\mathcal{M}_{\mathrm{script}}$ are executed by scripted controllers κ_m. The split is a property of the data-generation stage only: the deployed student receives neither the phase label as a switch nor separate networks per phase.

Let K be the required number of cycles, $\Delta\varphi^*$ the desired clockwise rotation per cycle, and φ_k the unwrapped nut angle about the bolt axis after the goal-reaching phase of cycle k, with φ_0 the angle at the start of the trial. The cyclic tracking error, the episodic tracking error, and the tracking cost of a trajectory τ are defined as

$$e_k = \varphi_k - \varphi_{k-1} - \Delta\varphi^*, \qquad e_{\mathrm{ep}} = \varphi_K - \varphi_0 - K\Delta\varphi^* = \sum_{k=1}^{K} e_k, \qquad E(\tau) = \frac{1}{K}\sum_{k=1}^{K} e_k^2 + \beta_E\, e_{\mathrm{ep}}^2. \tag{3.2}$$

Positive errors indicate overshoot and negative errors lag; the cost squares them so that overshoot in one cycle cannot cancel lag in another, and the weight β_E trades accumulated drift against cycle-wise accuracy. Let $C(\tau) \in \{0, \ldots, K\}$ count the cycles completed before failure or timeout and let $\Pi_{\mathcal{S}}$ denote the policies that execute the phases of $\mathcal{S}$ in order in every cycle. The research problem is to find the policy that chains all phases with maximum completion success and, among those, minimum tracking cost:

$$\Pi_{\mathrm{succ}} = \arg\max_{\pi \in \Pi_{\mathcal{S}}} \Pr_{\tau \sim p_\pi}[C(\tau) = K], \qquad \pi^* = \arg\min_{\pi \in \Pi_{\mathrm{succ}}} \mathbb{E}_{\tau \sim p_\pi}[E(\tau) \mid C(\tau) = K]. \tag{3.3}$$

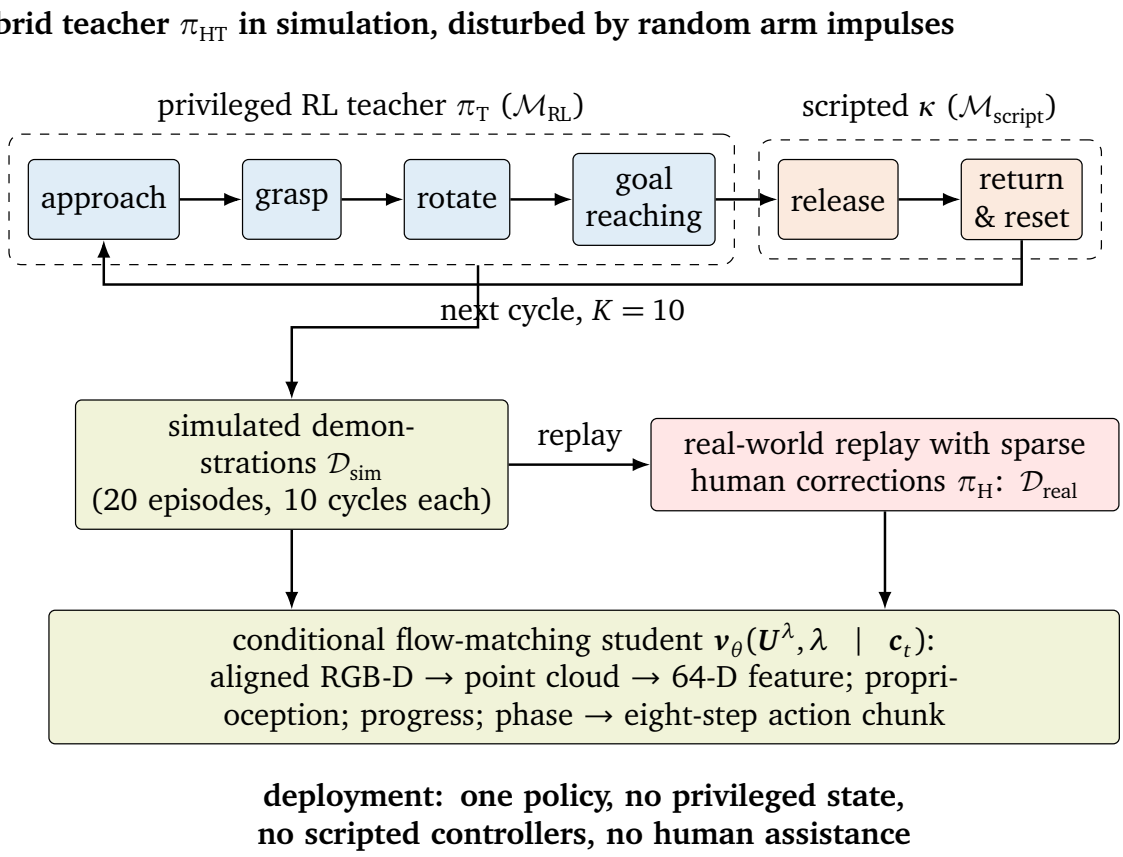


Figure 3.1: Overview of the pipeline. Decompose during data generation, unify at deployment: a privileged RL policy executes approach, grasp, rotation, and goal reaching, scripted controllers execute release and return, and together they form the hybrid teacher that generates multi-cycle demonstrations in simulation. Replaying the demonstrations on the robot with human corrections yields real-world demonstrations. Conditional flow matching distills the combined dataset into one visuomotor student that executes the complete cyclic task from RGB-D and proprioception.

Throughout this thesis $K = 10$ and $\Delta\varphi^* = 60°$, which corresponds to one face of a hexagonal nut per cycle. The policy π^* is learned from fixed hybrid-teacher demonstrations rather than by optimizing (3.3) on hardware, because on-robot optimization of a contact-rich cyclic task is neither safe nor sample-efficient.

The task is partially observable at deployment. The simulator state $\boldsymbol{x}_t$ contains the nut pose, the contact state, and the controller parameters; the student observes only an aligned RGB-D image, its proprioception, and two low-dimensional conditioning signals introduced in Section 3.5.3. The teacher–student structure of the method separates the two difficulties of this POMDP: the teacher solves the control problem with privileged state, and the student learns to perceive and to reproduce the teacher's behavior from sensor data.

3.2 Simulated and Physical System

3.2.1 Platform

The robot is a seven-DoF Franka Research 3 arm with an Inspire RH56DFX right hand mounted on the flange through a 10 mm adapter. The hand has twelve finger joints driven

by six motors; the non-thumb fingers each have one actuated flexion joint with a mechanically coupled distal joint, and the thumb has an abduction joint and a coupled flexion chain. Only three hand degrees of freedom are commanded by the policy: thumb abduction, thumb flexion, and index flexion, which together form the pinch used for threading; the remaining fingers hold a curled posture. The nut is threaded onto a bolt that stands on a 5 cm pedestal on a 60 cm by 80 cm bench; the robot base is mounted at tabletop height, 46.5 cm from the near table edge, so that the reach envelope lies above the work surface. A fixed Intel RealSense D415 provides the RGB-D observation; its extrinsic calibration relative to the robot base was measured on the physical cell and is reproduced in simulation.

Simulation uses Isaac Sim 5.1 with Isaac Lab 2.3 on the PhysX backend. Physics runs at 120 Hz; both teacher and student act at 15 Hz, so one policy command is held over eight physics substeps while the low-level controller closes its loop at the physics rate. The platform was selected over the Tiangong 2 humanoid arm on which the project started because the FR3 is substantially more precise and repeatable on the physical system; the task code, rewards, observation contracts, and training recipe are identical for both robots, and only the robot asset, workcell geometry, and reset poses differ.

3.2.2 Simplified Thread Simulation

Simulating a mating thread with mesh colliders is prohibitively expensive for the thousands of parallel environments that PPO needs. More importantly, two independent lines of experiments in this project showed that faithful signed-distance-field (SDF) thread colliders admit exploits: an RL policy reliably discovered that pressing the nut axially through the thread earns the descent reward without turning, and it did so under every tested depenetration setting and reward gate. The thread model used for teacher training therefore replaces the thread contact by a kinematic coupling, illustrated in Figure 3.2. The nut is attached to the bolt by a passive revolute joint about the bolt axis and a coaxial, position-controlled prismatic joint. With prismatic coordinate d_t, unwrapped and direction-corrected revolute coordinate θ_t, thread pitch ρ, and the index b_k of the step at which the current turn started, the prismatic target is

$$d_t^* - d_{b_k} = -\frac{\rho}{2\pi}\left(\theta_t - \theta_{b_k}\right), \tag{3.4}$$

so that one revolution produces one pitch of axial travel. The prismatic joint tracks d_t^* with a stiff drive (stiffness 2×10^5 N/m, damping 200 N s/m, force limit 1000 N), whose finite stiffness permits small deviations from the ideal helix while the revolute–prismatic pair constrains lateral motion and keeps the nut near the commanded path. The finger–nut contact forces remain fully simulated: the policy must physically grip and turn the nut. Thread resistance is modeled on the revolute joint by a smoothed Coulomb and viscous friction torque

$$\bar{\tau}_{f,t} = -\tau_c \tanh\left(\frac{\omega_t}{\omega_s}\right) - c_v\,\omega_t, \tag{3.5}$$

with revolute velocity ω_t, Coulomb torque $\tau_c = 2 \times 10^{-3}$ N m, viscous coefficient $c_v = 2 \times 10^{-4}$ N m s/rad, and smoothing speed $\omega_s = 0.1$ rad/s. The nut carrier has a mass of 1 g, and the joint limits reserve one full turn above the reset pose and a margin above the physical lower stop. The pitches are 3.0 mm, 3.5 mm, and 4.0 mm for the M24, M30, and M36 nuts. The model has one deliberate simplification beyond contact: the nut is initialized already engaged with the bolt. Attempts to learn or demonstrate thread engagement from a loose nut with reinforcement learning, teleoperation, and imitation learning were unsuccessful, which the thesis attributes to the narrow entry configuration that a rigid nut and a helical constraint admit.

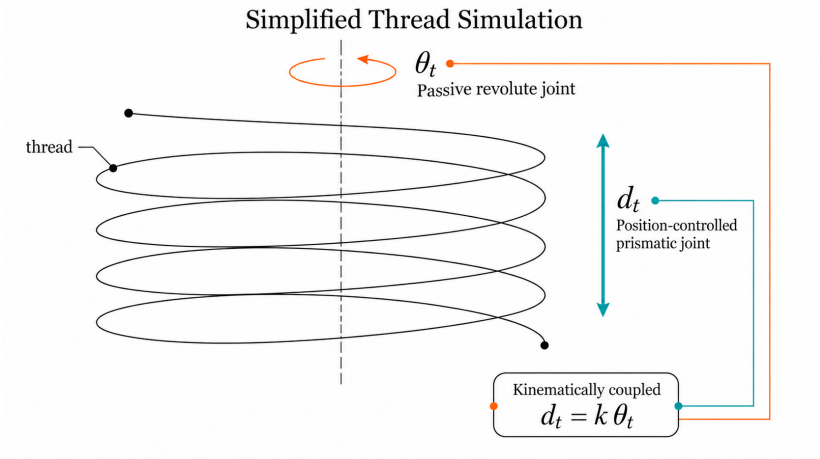


Figure 3.2: Simplified thread simulation. A passive revolute joint θ_t about the bolt axis is kinematically coupled to a position-controlled prismatic joint d_t with $d_t = k\theta_t$ and $k = -\rho/(2\pi)$, so that the nut advances by one pitch per revolution without explicit thread contacts. Figure reproduced from the accompanying paper [**NUTS2026**].

3.2.3 Action Contract and Controller

Teacher and student share one action space, a 9-D vector $\boldsymbol{a}_t \in [-1,1]^9$ that is decoded into a Cartesian target for the hand's grasp frame and three pinch-joint targets. The grasp frame is a fixed offset from the flange whose origin lies at the thumb–index pad midpoint, whose z axis points along the approach direction, and whose x axis is the pinch-closing axis. The first three components scale by 0.05 m to a position offset relative to an action frame that is the bolt tip during threading and the held nut during pickup; because the target is expressed relative to the bolt, the controller draws the grasped nut toward the bolt regardless of where nut and robot are, and a zero action does not hold the current pose. Components four to six set the orientation: roll and pitch scale by 1 rad around a neutral grasp orientation, and yaw maps $[-1,1]$ to the window $[-180°, 90°]$ inside the wrist joint limits. Components seven to nine command the thumb abduction, thumb flexion, and index flexion joints, where zero holds the closed-grasp posture and ± 1 moves by half the admissible joint range. The raw action is smoothed by an exponential moving average with factor sampled per episode from $[0.025, 0.1]$ (three times larger, capped at one, for the fingers), and the resulting target is clipped to at most 0.02 m and 0.097 rad from the current pose per step, so that one command cannot demand a large jump.

The Cartesian target is tracked by the task-space impedance controller of (2.3), inherited from Factory and FORGE, with proportional gains $[565, 565, 565, 28, 28, 28]$ (N/m and N m/rad), critical damping, a nullspace posture term with gains 10 and 6.32, and a dead zone of $[5, 5, 5, 1, 1, 1]$ (N and N m) below which wrench components are zeroed. The gains act component-wise in world axes, so the impedance ellipsoid is aligned with the vertical bolt axis. The arm's own joint drives are disabled and the controller output is applied as joint torque; the finger joints are position-controlled by the simulator's implicit PD drives. For training robustness and for sim-to-real transfer, the controller parameters are randomized per episode: gains by a multiplicative factor in $[1/1.41, 1.41]$, position and rotation thresholds by $\pm 25\%$ and $\pm 29\%$, and the dead zone by uniform resampling every two seconds. The critic observes the sampled values, the actor does not.

For deployment, the same controller law is reproduced on the physical FR3 in a libfranka-style Cartesian impedance controller that uses the quaternion-vector orientation error, a damped pseudoinverse for the nullspace projector, and the robot's own gravity and Coriolis compensation; the gains of the physical controller are documented in Chapter 5. This

choice of a shared Cartesian action space for teacher, student, and robot is the result of a design decision that is worth recording, because the project also implemented and evaluated an alternative. The alternative, called the 17-D impedance joint-PD contract, has the student emit seven arm position references, seven arm velocity references, and three pinch positions, which a joint-space impedance controller tracks as

$$\boldsymbol{\tau} = \boldsymbol{g}(\boldsymbol{q}) + K_p^{\mathrm{hw}}(\boldsymbol{q}_{\mathrm{ref}} - \boldsymbol{q}) + K_d^{\mathrm{hw}}(\dot{\boldsymbol{q}}_{\mathrm{ref}} - \dot{\boldsymbol{q}}), \tag{3.6}$$

with gravity compensation $\boldsymbol{g}(\boldsymbol{q})$ owned by the controller. Teacher labels are converted to this contract by evaluating the Cartesian controller and reverse-engineering the equivalent joint references, $\boldsymbol{q}_{\mathrm{ref}} = \boldsymbol{q} + (K_p^{\mathrm{hw}})^{-1}\boldsymbol{\tau}_{\mathrm{OSC}}$ and $\dot{\boldsymbol{q}}_{\mathrm{ref}} = \dot{\boldsymbol{q}}$. The motivation was that a joint-space command is the native interface of most robot drivers. Replaying recorded 17-D commands at the physics rate reproduced the teacher motion with an arm-joint RMSE of 0.0037 rad, so the controller itself is adequate. The contract is nevertheless unfavorable for learning: the equivalent joint target changes within the eight physics ticks over which a 15 Hz student command is held, so a zero-order hold introduces a target error of 0.010 to 0.015 rad and an implied torque error of up to 16 N m; the commands are in physical units with widely different scales across the 17 dimensions; and, as Chapter 4 reports, offline students trained on this contract reproduced their training labels but did not reach the task goals in closed loop. The Cartesian contract, in which the student imitates the bounded, bolt-relative action that the teacher itself produced and executes it through the unchanged controller, removed this conversion and is the contract of all reported results. The 17-D contract is retained as an ablation.

3.3 Privileged Teacher Training

The teacher for the learned phases in $\mathcal{M}_{\mathrm{RL}}$ is a recurrent policy trained with PPO in an asymmetric actor–critic. The actor observation $\boldsymbol{z}_t^{\pi} = g_{\pi}(\boldsymbol{x}_t, \boldsymbol{\xi}_t)$ selects the actor-visible components of the simulator state and applies observation noise $\boldsymbol{\xi}_t$; it comprises the fingertip (grasp-frame) position relative to the bolt frame and its quaternion, the end-effector linear and angular velocity obtained by finite differences, the fingertip position relative to the held nut and the nut quaternion, the position and quaternion error to the current turning target, a grasp-phase flag that indicates whether the proper grasp has latched, the previous filtered action, and, for the multi-size teacher, a one-hot code of the nut size. Position terms receive Gaussian noise with 0.25 mm standard deviation per axis, orientation terms 0.1°, and the bolt frame a per-episode offset of 1 mm, which models calibration error. The critic observes the same task and history structure using clean values and extends it with the arm joint positions, the complete nut and bolt pose, the sampled controller gains, smoothing factor, and thresholds, and the thumb and index contact forces. Table A.1 in the appendix lists the exact components and widths.

Actor and central critic share the same architecture: a two-layer LSTM with 1024 units per layer and layer normalization, followed by a 512–128–64 MLP with ELU activations. Recurrent sequences span 128 control steps, the rollout horizon is 256 steps, and the actor parameterizes a diagonal Gaussian over the normalized action with a state-dependent standard deviation. The teacher is optimized with the shaped reward

$$r_t = \sum_{i \in \mathcal{R}} w_i^r R_{i,t} - \sum_{j \in \mathcal{P}} w_j^p P_{j,t},$$
$$\mathcal{R} = \{\mathrm{center}, \mathrm{face}, \mathrm{grasp}, \mathrm{turn}, \mathrm{target}\}, \qquad \mathcal{P} = \{\mathrm{mag}, \Delta a, \mathrm{align}, \mathrm{force}, \mathrm{settle}, \mathrm{clear}, \mathrm{contact}\}. \tag{3.7}$$

The reward follows the three phases of the learned behavior. During approach, R^{center} rewards motion of the grasp center toward the nut center and R^{face} rewards motion of the thumb and index pads toward the two designated opposing faces of the nut; both pay 20 times the newly achieved fraction of the initial gap, so repeated motion cannot accumulate reward. R^{grasp} pays a one-time bonus of 100 when a proper grasp, defined as simultaneous thumb and index contact on the designated faces with a pinch line perpendicular to the nut axis, has held for five consecutive steps. During turning, R^{turn} pays 60 times the newly achieved fraction of the distance to the fixed helical target pose, and pays only while the previously acquired proper grasp remains live. At completion, R^{target} pays 250 distributed over a five-step hold within 5° of the target. The penalties discourage large commands (P^{mag}, weight 10^{-3}), abrupt action changes ($P^{\Delta a}$, 0.1), misalignment of the hand frame with the nut frame (P^{align}, 0.02), excess wrist force (P^{force}, 0.05), residual rotation near the target (P^{settle}, $0.25\,\omega_t^2$ from 50° onward), insufficient table clearance (P^{clear}), and arm–table contact (P^{contact}). An episode succeeds when at least 55° of clockwise rotation is held with a live proper grasp for five steps; it terminates early when the nut leaves the bolt by more than 3 cm, tilts by more than 60°, or releases the thread, and when the grasp is lost after it has latched. The last rule turns grasp retention into a hard constraint rather than a shaped term.

Robustness is obtained by domain randomization of the nut mass (±5 g), the bolt friction (static coefficient in $[0.25, 1.25]$), the controller parameters of Section 3.2.3, and the initial state; Table 3.1 lists the ranges. The observation noise is small in absolute terms, a quarter of a millimeter and a tenth of a degree, because the actor's inputs are relative poses that the student will later infer from a calibrated camera; the per-episode bolt-frame offset of one millimeter is the term that models calibration error and forces the teacher to close the loop on the nut rather than on the bolt frame. The initial nut rotation is randomized by an automatic curriculum: the yaw range starts at zero and expands by 5° whenever the population success rate exceeds 80 %, up to ±30°, with at least five episode lengths between expansions; the initial axial height is sampled uniformly up to 20 mm below the nominal spawn while reserving room for the full 60° target turn.

Table 3.1: Domain randomization during teacher training. $U(a, b)$ denotes uniform sampling; multiplicative factors are sampled as $1 + U(0, \ell)$ or its reciprocal with equal probability.

Group	Quantity	Range
physics	nut mass offset	$U(-5, 5)$ g per reset
	bolt static friction	$U(0.25, 1.25)$, 128 buckets
controller	proportional gains	multiplicative, $\ell = 0.41$, per axis and reset
	action smoothing factor	$U(0.025, 0.1)$ per reset; fingers 3×
	position / rotation threshold	multiplicative, $\ell = 0.25$ / 0.29
	dead zone	$U(0, [5, 5, 5, 1, 1, 1])$ N, N m, resampled every 2 s
observation	fingertip and nut position	$\mathcal{N}(0, 0.25\,\text{mm})$ per axis and refresh
	fingertip and nut orientation	random-axis angle $\mathcal{N}(0, 0.1°)$
	bolt frame	$\mathcal{N}(0, 1\,\text{mm})$ per axis and reset
	fingertip forces	$\mathcal{N}(0, 1\,\text{N})$ per channel
initial state	nut yaw	curriculum $0 \rightarrow \pm 30°$ in 5° steps at 80 % success
	nut axial height	$U(0, 20)$ mm below nominal, full turn reserved
	nut size	round-robin M24 / M30 / M36 (multi-size teacher)

A single teacher is conditioned on the M24, M30, and M36 nuts through the one-hot code and round-robin assignment of sizes to environments. Training uses 512 parallel environments for 200 epochs and takes about ten hours on one NVIDIA GeForce RTX 5090; the PPO hyperparameters are listed in Table A.2 in the appendix.

A second expert, the pickup teacher, was trained with the same recipe for grasping a loose

nut from the table, lifting it, transporting it to a handoff pose above the bolt tip, and engaging it; its reward uses the same approach and grasp terms, a lift term, and keypoint progress toward two waypoints. It is part of the full-task variant of the hybrid teacher described below and is reported for completeness in Chapter 4; it is not part of the final threading pipeline, whose trials start from an engaged nut.

3.4 Hybrid Teacher and Demonstration Generation

3.4.1 Cyclic Teacher Rollout in Simulation

Let $m_t \in \mathcal{M}$ denote the active phase, $\boldsymbol{x}_t$ the full simulator state, $\boldsymbol{z}_t^{\pi}$ the noisy actor observation, and $\boldsymbol{h}_t$ the actor's recurrent state. During rollout the teacher is queried for its deterministic mean action $\bar{\pi}_{\mathrm{T}}(\boldsymbol{z}_t^{\pi}, \boldsymbol{h}_t) = \mathbb{E}_{\boldsymbol{a}\sim\pi_{\mathrm{T}}(\cdot|\boldsymbol{z}_t^{\pi},\boldsymbol{h}_t)}[\boldsymbol{a}]$. The hybrid teacher is

$$\pi_{\mathrm{HT}}(\boldsymbol{x}_t, m_t) = \begin{cases} \bar{\pi}_{\mathrm{T}}(\boldsymbol{z}_t^{\pi}, \boldsymbol{h}_t), & m_t \in \mathcal{M}_{\mathrm{RL}}, \\ \kappa_{m_t}(\boldsymbol{x}_t), & m_t \in \mathcal{M}_{\mathrm{script}}, \end{cases} \tag{3.8}$$

where κ_{m_t} is the scripted controller of the active transition phase. Both branches produce commands in the same filtered, bolt-relative Cartesian action space, so the recorded label is a single 9-D signal throughout the cycle; the scripted Cartesian targets are inverse-decoded through the action mapping of Section 3.2.3, and the executing adapter compensates the action smoothing analytically so that the controller receives exactly the requested target. This unified label space is what makes the demonstration a coherent supervision signal for one student: at no step does the label change meaning or units.

Release is triggered when the nut rotation lies within $\pm 5^{\circ}$ of the 60° target while a valid opposed-finger grasp has been maintained for a short dwell. The fingers then open to a safe clearance, after which four Cartesian waypoints retract the hand along the ray from the nut center through the grasp center, raise it, and return it to the initial grasp pose. Each waypoint segment is interpolated over 0.7 s and the final return over 1.0 s; the return is physically gated and counts as complete only when the grasp frame is within 5 mm and 5° of the reset frame and the hand joints within 0.15 rad of their reset values, with a 16 s timeout that marks the cycle as a transition failure. The waypoints were authored once by a collision-safe retargeting from a captured release pose and verified to produce zero pad–nut contact force during every release. While the hand is away from the nut the thread joint is held, so the nut cannot drift between cycles. On a completed return the turning reference is rebased: the target angle advances by 60° relative to the current nut pose, the frozen threading policy receives a cycle-local canonicalized nut orientation so that it sees the same observation statistics in every cycle, and its recurrent state and action history are reseeded. The physical nut pose is not reset, so the demonstration contains the axial advance of the nut over ten cycles. The teacher population was first qualified for one period only; repeated cycles are a property of the hybrid rollout, not of the RL teacher alone. Figure 3.3 shows the coordinator that sequences one cycle.

Two details of the coordinator matter for the quality of the labels. First, the scripted transitions never write physics state: a transition may request that the thread joint be held while the hand is away, but it repairs nothing by teleporting the nut, so every recorded frame is a physically consistent consequence of the recorded commands; an earlier implementation that snapped the nut back at the policy rate produced visible reversals of the nut at release that the student would have imitated. Second, the release gate is physical rather than timed: release is decided from the measured nut rotation and the measured grasp, and the return is

validated against the reset frame, so the phase labels in the recording coincide with physical events. Both properties are what allow the recorded phase to be used as a conditioning signal at deployment, where it is derived from the same physical events.

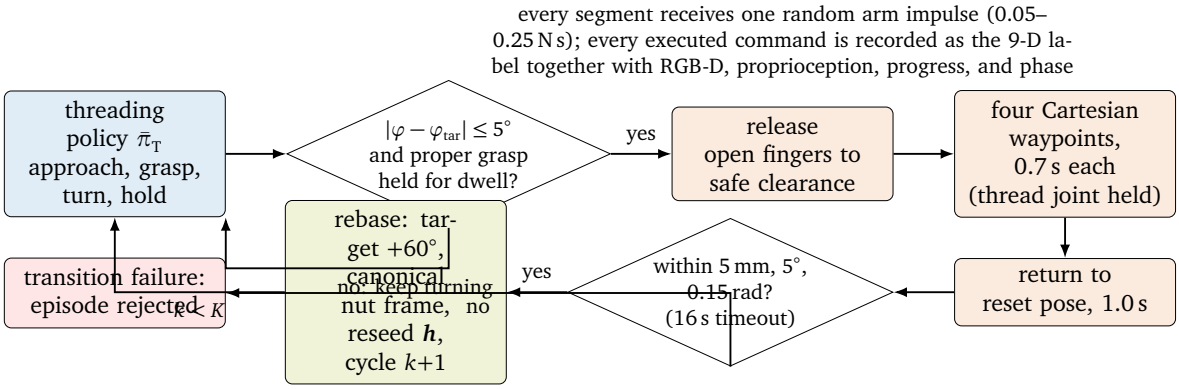


Figure 3.3: Data-collection loop of the hybrid teacher for one threading cycle. The learned policy owns approach, grasp, turn, and hold; a physical release gate hands over to the scripted release, four retreat waypoints, and the gated return; a completed return rebases the turning target and starts the next cycle. Every executed command is recorded in the common 9-D label space.

3.4.2 Full-Task Variant with Pickup and Handover

The hybrid teacher also has a full-task variant that prepends the pickup expert and a scripted pickup-to-threading handover (open the hand, move to the threading reset pose, close, settle, with phase durations of 0.4, 1.5, 0.4, and 0.25 s) to the cyclic threading teacher. The resulting demonstrations start from a loose nut on the table. Both variants and their single-phase special cases (pickup only, one threading turn, cyclic threading) are selected by one training-environment option and expose the same supervision contract, so that a student trained on either variant can be evaluated with the same tools. The full-task variant depends on the pickup expert reaching its engagement goal reliably, which was not achieved (Chapter 4); the reported threading results therefore use the cyclic variant from an engaged nut.

3.4.3 Disturbance Augmentation

Twenty demonstrations contain too little variation for a student that must recover from its own deviations. Trajectory diversity is therefore increased at collection time by applying random impulses to the arm: in every contiguous teacher-phase segment, that is, in every policy, waypoint-following, and return segment of every cycle, one impulse with a magnitude of 0.05 to 0.25 N s is applied to a random arm link between 0.05 and 0.20 s after the segment begins, giving 30 impulses per ten-cycle demonstration. The teacher, which is a closed-loop controller, reacts to the disturbance, and the recorded recovery is part of the label. Because the teacher runs deterministically at $\beta = 1$ with the disturbance as the only source of variation, the recovery segments remain consistent with the trajectory that conditions them. Demonstrations in which the teacher fails to complete all cycles are rejected before materialization; the retained set consists only of explicitly successful episodes.

3.4.4 Human-Corrected Real-World Replay

Simulated demonstrations are replayed on the physical robot through the same Cartesian controller. Replay exposes the sim-to-real discrepancies of the simplified thread model and of the real contact: the fingers may slip on the nut faces, or the grasp may be misaligned so that the turn does not engage. Whenever such a failure is anticipated, a human operator takes over and replaces the affected teacher-action segment from grasp through release, recovering stable contact and completing the turn; the operator's actions overwrite the corresponding teacher actions in the trajectory, and the corrected trajectory is recorded as a real-world demonstration. The intervention is sparse by design, because the operator acts only in the grasp-to-release segment and only when a failure is anticipated. The set $\mathcal{D}_{\text{real}}$ of corrected replays, together with the simulated set $\mathcal{D}_{\text{sim}}$, forms the fixed offline dataset $\mathcal{D} = \mathcal{D}_{\text{sim}} \cup \mathcal{D}_{\text{real}}$ used for distillation. The real-world observations are the camera stream and proprioception of the physical system, so the student is exposed to real sensor statistics without any real-world reinforcement learning.

3.4.5 Dataset Format

Each demonstration is materialized as one episode file with aligned 15 Hz samples. A sample holds the RGB image and the metric depth image at 320×180 pixels, obtained by center-cropping the 640×480 camera stream to 640×360 and downscaling by two with the intrinsics scaled accordingly; the 29-D proprioception, consisting of the ten controlled joint positions, their velocities, and the previous filtered 9-D action; the executed 9-D command; the normalized task progress; the active process phase; the nut pose for privileged ablations and auxiliary supervision; and the timestamp. The recorder stores the 17-D joint-PD equivalent of every command alongside, so that both action contracts can be trained from one collection. Chunks that cross a physical reset are masked, phase boundaries are retained so that the student learns the transitions, and terminal chunks are padded with the final valid command so that the last observations of an episode remain conditioning states. For the retained M24 data set, 20 successful ten-cycle demonstrations collected from 32 parallel environments contain 19 486 frames.

3.5 Student Policy Distillation by Conditional Flow Matching

3.5.1 From Behavior Cloning to Online Distillation for Cyclic Tasks

Student learning is posed as supervised imitation from the fixed dataset $\mathcal{D}$. Section 2.1.2 explained why plain behavior cloning is fragile over long horizons: the student is evaluated on the states that its own errors produce. For cyclic threading the mechanism is concrete. A student that grasps a millimeter too high applies a slightly different torque, the nut advances a little less, and after a few cycles the fingers close on the edge of the nut instead of on its faces. Three measures counter this compounding in the present method, and they are complementary. First, the demonstrations themselves contain recoveries, because the closed-loop teacher was disturbed by impulses during collection; the student thus sees off-nominal states with correct labels, which is the effect DAgger produces by querying the expert on student-visited states, obtained here without an expert that can be queried at arbitrary states. Second, the student predicts action chunks and executes a temporal ensemble of overlapping chunks, which filters the single-step noise that would otherwise be amplified by the controller.

Third, where a queryable expert exists, in simulation, the same flow-matching student can be trained online with the hybrid teacher labeling the states that the student visits; this online variant is described in Section 3.5.5 and compared with the offline variant in Chapter 4. The decisive practical constraint is the real-world data: the human-corrected replays are an offline dataset, and they are the data that close the sim-to-real gap. The deployed student is therefore trained offline on $\mathcal{D}$, and online DAgger serves as a simulation-only ablation.

3.5.2 Flow Matching for Conditional Action Chunks

Flow matching learns a generative model as the solution of an ODE. Let $\boldsymbol{x} \in \mathbb{R}^d$ denote a sample, let $p_0 = \mathcal{N}(\mathbf{0}, I)$ be the source distribution, and let $p_1 = q$ be the data distribution. A time-dependent vector field $\boldsymbol{v} : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ defines a flow ψ_λ through $\frac{\mathrm{d}}{\mathrm{d}\lambda}\psi_\lambda(\boldsymbol{x}) = \boldsymbol{v}(\psi_\lambda(\boldsymbol{x}), \lambda)$ with $\psi_0(\boldsymbol{x}) = \boldsymbol{x}$, and the flow transports p_0 to the marginal path $p_\lambda = [\psi_\lambda]_{\#} p_0$. The flow-matching objective regresses a parametric field $\boldsymbol{v}_\theta$ onto a target field $\boldsymbol{u}_\lambda$ that generates a prescribed path p_λ from p_0 to p_1:

$$L_{\mathrm{FM}}(\theta) = \mathbb{E}_{\lambda \sim U[0,1],\, \boldsymbol{x} \sim p_\lambda} \left\| \boldsymbol{v}_\theta(\boldsymbol{x}, \lambda) - \boldsymbol{u}_\lambda(\boldsymbol{x}) \right\|_2^2. \tag{3.9}$$

This objective cannot be evaluated directly, because the marginal path and its field are defined only implicitly by the data. Conditional flow matching [Lip+23] resolves this by constructing the marginal path as a mixture of conditional paths $p_\lambda(\boldsymbol{x} \mid \boldsymbol{x}^1)$, one per data sample $\boldsymbol{x}^1 \sim q$, each with a known conditional field $\boldsymbol{u}_\lambda(\boldsymbol{x} \mid \boldsymbol{x}^1)$. The marginal field is the posterior-weighted average of the conditional fields,

$$\boldsymbol{u}_\lambda(\boldsymbol{x}) = \int \boldsymbol{u}_\lambda(\boldsymbol{x} \mid \boldsymbol{x}^1) \, \frac{p_\lambda(\boldsymbol{x} \mid \boldsymbol{x}^1) q(\boldsymbol{x}^1)}{p_\lambda(\boldsymbol{x})} \, \mathrm{d}\boldsymbol{x}^1, \tag{3.10}$$

and the conditional objective

$$L_{\mathrm{CFM}}(\theta) = \mathbb{E}_{\lambda \sim U[0,1],\, \boldsymbol{x}^1 \sim q,\, \boldsymbol{x} \sim p_\lambda(\cdot \mid \boldsymbol{x}^1)} \left\| \boldsymbol{v}_\theta(\boldsymbol{x}, \lambda) - \boldsymbol{u}_\lambda(\boldsymbol{x} \mid \boldsymbol{x}^1) \right\|_2^2 \tag{3.11}$$

has the same gradient with respect to θ as (3.9); the two differ only by a constant that does not depend on θ, which follows by expanding the squares and using (3.10). The conditional path can therefore be chosen for convenience. The optimal-transport path takes the conditional distribution to be Gaussian with mean $\lambda \boldsymbol{x}^1$ and standard deviation $1 - \lambda$ (up to a small floor), so that a sample is the straight-line interpolant

$$\boldsymbol{x}^\lambda = (1-\lambda)\boldsymbol{x}^0 + \lambda \boldsymbol{x}^1, \qquad \boldsymbol{x}^0 \sim p_0, \tag{3.12}$$

and the conditional field along this line is constant, $\boldsymbol{u}_\lambda(\boldsymbol{x}^\lambda \mid \boldsymbol{x}^1) = \boldsymbol{x}^1 - \boldsymbol{x}^0$. Straight conditional paths make the learned marginal field nearly straight as well, which is why few integration steps suffice at sampling time.

The student applies this construction to action chunks conditioned on the observation. At control step t the student observes $\boldsymbol{y}_t = (\boldsymbol{O}_t^{\mathrm{RGBD}}, \boldsymbol{s}_t, p_t, \boldsymbol{e}_t^m)$, consisting of the RGB-D image, the proprioception $\boldsymbol{s}_t$, the normalized task progress p_t, and the process-phase encoding $\boldsymbol{e}_t^m$, and learns the distribution of the executed H-step action chunk. The supervision target is the demonstrated chunk $\boldsymbol{U}_t^D = [\boldsymbol{u}_t, \ldots, \boldsymbol{u}_{t+H-1}] \in \mathbb{R}^{H \times 9}$, the observation is encoded into a condition vector $\boldsymbol{c}_t$ by the network of Section 3.5.3, and the distillation objective is

$$\theta^* = \arg\min_\theta \, \mathbb{E} \left\| \boldsymbol{v}_\theta\big(\boldsymbol{U}_t^\lambda, \lambda \mid \boldsymbol{c}_t\big) - \dot{\boldsymbol{U}}_t^\lambda \right\|_2^2, \qquad \boldsymbol{U}_t^\lambda = (1-\lambda)\boldsymbol{U}_t^0 + \lambda \boldsymbol{U}_t^D, \quad \dot{\boldsymbol{U}}_t^\lambda = \boldsymbol{U}_t^D - \boldsymbol{U}_t^0, \tag{3.13}$$

where the expectation is over $(\boldsymbol{y}_t, \boldsymbol{U}_t^D) \sim \mathcal{D}$, $\boldsymbol{U}_t^0 \sim \mathcal{N}(\mathbf{0}, I)$, and $\lambda \sim U[0,1]$. The learned field transports Gaussian noise to the conditional action distribution $p_\theta(\boldsymbol{U}_t \mid \boldsymbol{y}_t)$. At inference, a chunk is generated by drawing $\boldsymbol{U}^0 \sim \mathcal{N}(\mathbf{0}, I)$ and integrating the ODE with the explicit Euler method on a uniform grid of N steps,

$$\boldsymbol{U}^{\lambda_{n+1}} = \boldsymbol{U}^{\lambda_n} + \frac{1}{N}\, \boldsymbol{v}_\theta\big(\boldsymbol{U}^{\lambda_n}, \lambda_n \mid \boldsymbol{c}_t\big), \qquad \lambda_n = \frac{n}{N}, \quad n = 0, \ldots, N-1, \tag{3.14}$$

with $N = 16$ at deployment. The condition $\boldsymbol{c}_t$ is computed once per chunk, so the image is encoded once and the N velocity evaluations cost only the Transformer of Section 3.5.3.

Three properties distinguish this head from the alternatives that the thesis compares against. Relative to a deterministic regression head, which minimizes a squared error between a single predicted chunk and the label and therefore converges to the conditional mean, the flow head represents multimodal labels; this matters because the dataset mixes simulated teacher actions with human corrections for the same observation, and because the teacher's own recoveries after impulses are state-dependent in ways the observation does not fully resolve. Relative to a diffusion head, which learns the score of a noised data distribution and samples by a stochastic reverse process with many steps along curved paths, the flow head learns a deterministic ODE along nearly straight paths, trains with a plain regression target without a noise schedule, and samples in sixteen steps. The remaining stochasticity of the flow head comes from the source sample only; a zero-noise sample at the source mean is available as a diagnostic.

3.5.3 Student Architecture as Implemented

Figure 3.4 shows the student. It consists of a vision encoder, a proprioception encoder, a fusion and conditioning stage, and the velocity network.

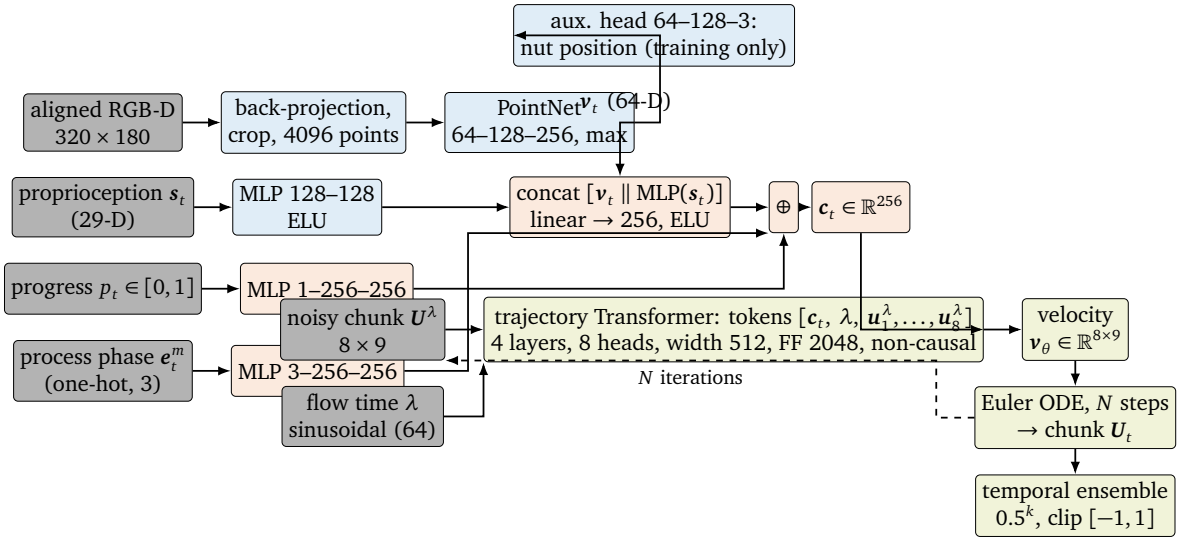


Figure 3.4: Student architecture. The aligned RGB-D observation is back-projected into a point cloud and encoded into a 64-D visual feature; the 29-D proprioception is encoded by an MLP; late fusion concatenates the two with the visual feature first and projects to the 256-D condition, to which learned embeddings of the global trajectory progress and of the process phase are added. A non-causal trajectory Transformer predicts the flow velocity of all eight action tokens given the condition token and a sinusoidal embedding of the flow time. An auxiliary head regresses the nut position from the visual feature during training.

Observation encoding and late fusion. Following DextrAH-RGB [Sin+25], the visual and proprioceptive streams are encoded separately and fused by concatenation, with the visual feature placed before the encoded proprioception. The 29-D proprioception $\boldsymbol{s}_t$, consisting of the ten controlled joint positions, the ten joint velocities, and the previous filtered 9-D action, is encoded by a 128–128 MLP with ELU activations. The concatenation of the 64-D visual feature and the 128-D proprioceptive feature is projected by a linear layer with ELU activation to the 256-D condition. In the privileged ablations of Chapter 4 the vision encoder is replaced by the identity and the nut position (3-D) or the nut position with its canonical axis-angle orientation (6-D) is appended to the proprioception instead; in the ablation that removes the hand velocities, the three pinch-joint velocities are dropped and the proprioception becomes 26-D.

Vision encoder. The retained encoder follows DP3 [Ze+24]. The depth image is back-projected through the calibrated intrinsics into camera-frame points; points with non-finite depth, depth outside $[0.10, 2.0]$ m, or coordinates outside a fixed crop box that covers the workspace with a 40 mm margin are rejected; the remaining points are centered and scaled by fixed workspace constants and carry the color of their pixel, normalized to $[-1, 1]$. From the valid points, 4096 are sampled, randomly during training and with a fixed priority at evaluation; training fails fast if fewer than 128 valid points remain. A compact PointNet with pointwise layers of width 64, 128, and 256, layer normalization, ReLU activations, and global max pooling, followed by an ELU projection, yields the 64-D visual feature $\boldsymbol{v}_t$. The encoder has about 60 000 parameters and no pretrained backbone; it can be initialized from a pose-regression pretraining on rendered sweeps of a free-moving nut, which Chapter 4 evaluates. Two alternatives are implemented with the same 64-D interface. An image-space encoder applies an ImageNet-pretrained ResNet-18 stage-2 trunk to the concatenation of RGB, log depth, a validity mask, and the calibrated camera-frame XYZ of each pixel, followed by one spatial self-attention layer and learned-query pooling; this is the ablation (b) of Chapter 4. An Adapt3R-style encoder [**Wilcox2025**] extracts dense features with a frozen ResNet-18 layer-1 backbone, localizes 512 of them in 3D through the depth map, expresses the points in the live grasp frame, and pools them by learned-query attention. All encoders consume the same 320×180 RGB-D pair.

Conditioning on progress and phase. Two scalar-sized conditions are added to the 256-D condition through separate learned embeddings, each a 1–256–256 or 3–256–256 MLP with ELU activation. The global trajectory progress $p_t \in [0, 1]$ maps the first timestamp of each demonstration to 0 and the last to 1; at deployment it advances with elapsed time from the start of the trial, normalized by the median demonstration duration stored in the checkpoint, and is clamped to 1 afterwards. It is a monotonic clock for the complete behavior and nothing more: it is not a teacher-stage selector, it is not periodic, and it does not encode the cycle index; it tells the student how far into a nominally timed trial it is, which disambiguates, for instance, the first and the last approach of a trial that otherwise look alike. The process-phase encoding $\boldsymbol{e}_t^m$ is a three-way one-hot over the policy-controlled phases, the release waypoints, and the return; during training it is derived from the recorded phase labels, and during evaluation it is published by the cyclic coordinator from physical events, so that a cycle that takes longer than nominal cannot select the wrong phase. The phase encoding is a synchronization aid: it does not switch networks or action contracts, and the student predicts the transition actions itself.

Velocity network. The velocity field is a non-causal trajectory Transformer. The noisy chunk $\boldsymbol{U}^\lambda$ is split into $H = 8$ action tokens of dimension 9, each embedded linearly into

the 512-D model width; a condition token embeds $\boldsymbol{c}_t$ and a time token embeds the flow time λ through a 64-D sinusoidal encoding. Learned positional embeddings mark the ten token positions; they encode only the offset of an action inside the chunk. Four pre-norm Transformer encoder layers with eight attention heads, a 2048-D GELU feed-forward block, and no dropout apply full self-attention over all tokens, so that every future step may attend to the whole noisy chunk when predicting its velocity, and a linear output layer maps each action token to its 9-D velocity. The model keeps an exponential moving average of its parameters with decay 0.999, and inference uses the averaged weights. An auxiliary head, a 64–128–3 MLP on the visual feature, regresses the nut position in normalized coordinates with unit weight; it regularizes the visual feature toward spatially meaningful content and is discarded at deployment. Training uses the Adam optimizer with learning rate 10^{-4}, batches of 64 windows, 300 epochs, and DextrAH-style RGB augmentation of the stored images, with saturation, contrast, and brightness factors in $[0.5, 1.5]$, hue shifts in $[-0.15, 0.15]$, and motion blur with probability 0.1.

Action chunking and execution. With $H = 8$ and 15 Hz, one chunk covers 0.53 s. Chunk targets begin at the sample paired with the observation (target offset zero), which is the correct alignment for the Cartesian contract because the recorder stores the pre-step state together with the command acting on it. At deployment the student replans at every control step, and the command sent to the controller is a temporal ensemble of all still-valid proposals for the current tick with weights 0.5^k for a proposal of age k; a decay of zero recovers strict first-action execution. The ensemble operates in action space and is distinct from the parameter-space EMA. The executed 9-D action is clipped to $[-1, 1]$ and passes through the unchanged controller path of Section 3.2.3, including the exponential action smoothing. Inference at 15 Hz with 16 Euler steps on an RTX 5090 fits within the control period; the measured latency is TBD.

Normalization. The Cartesian actions are already normalized to $[-1, 1]$ by the action contract, so no action statistics are learned. Proprioceptive inputs are consumed in physical units; the nut position of the auxiliary head is standardized by dataset statistics stored in the checkpoint.

3.5.4 Offline Training

Algorithm 1 summarizes offline distillation. The dataset is formed from simulated rollouts and human-corrected replays, windows of H consecutive commands are built from each episode without crossing episode boundaries, and the velocity field is trained with (3.13) while an EMA copy of the parameters is maintained.

3.5.5 Online DAgger with a Flow-Matching Head

The online variant replaces the fixed dataset by live teacher labels. The simulator is driven by a mixture of teacher and student control with teacher probability β, the hybrid teacher labels every visited state with its 9-D command, and the flow head is trained on the labels as they arrive. Two aspects differ from the recurrent DAgger of DextrAH and of the predecessor thesis. First, the label of a chunked student is a sequence of H future teacher commands, not a single command. The trainer collects eight consecutive live teacher commands, pairs the sequence with the observation at its first step, and uses stride-one overlapping supervision, so

Algorithm 1 Offline flow distillation with human intervention

Require: hybrid teacher π_{HT}, human operator π_{H}, horizon H
1: Collect simulated rollouts $\mathcal{D}_{\mathrm{sim}}$ with π_{HT} under random arm impulses
2: Replay them on the robot, overwriting failure-prone grasp-to-release segments with π_{H}, to obtain $\mathcal{D}_{\mathrm{real}}$
3: Form H-step chunks from $\mathcal{D} = \mathcal{D}_{\mathrm{sim}} \cup \mathcal{D}_{\mathrm{real}}$, masking resets and padding terminal chunks
4: **repeat**
5: Sample $(\boldsymbol{y}_t, \boldsymbol{U}_t^D) \sim \mathcal{D}$, $\boldsymbol{U}_t^0 \sim \mathcal{N}(\mathbf{0}, I)$, $\lambda \sim U[0,1]$
6: Encode the point cloud and fuse it with proprioception, progress, and phase to obtain $\boldsymbol{c}_t$
7: Update θ by one gradient step on (3.13) plus the auxiliary nut-position loss
8: Update EMA parameters $\bar{\theta} \leftarrow \alpha\bar{\theta} + (1-\alpha)\theta$
9: **until** convergence
10: **return** student velocity field $\boldsymbol{v}_{\bar{\theta}}$

that once the initial horizon is full one chunk completes at every control step; sequences that cross a reset or the scripted handover are discarded. Second, the chunk label is only a valid sample of $p(\boldsymbol{U}_t \mid \boldsymbol{y}_t)$ if the teacher itself produced the states on which the later commands were computed. If the student had controlled some of those steps, the later labels would describe the teacher's reaction to a student-induced future, not the teacher's trajectory from the conditioning observation. Multi-step online supervision therefore fixes $\beta = 1$ during collection: the teacher carries every training state, and the student is exposed to its own state distribution only at evaluation. The RGB-D latent is encoded once per step and detached before it enters the pending windows; the vision encoder is trained by the immediate nut-position auxiliary objective at a reduced learning rate of 10^{-5}, and the delayed flow objective trains the action path from the stored latent with learning rate 10^{-4}. Sampling during online evaluation uses eight Euler steps. Algorithm 2 summarizes the loop.

Algorithm 2 Online DAgger with a conditional flow-matching head

Require: hybrid teacher π_{HT}, student $\boldsymbol{v}_\theta$, horizon H, teacher probability β
1: **for** each iteration **do**
2: Observe $\boldsymbol{y}_t$ in every environment; query the teacher label $\boldsymbol{u}_t^* = \pi_{\mathrm{HT}}(\boldsymbol{x}_t, m_t)$
3: Sample a student chunk from $\boldsymbol{v}_\theta$ with N Euler steps and take its first action $\hat{\boldsymbol{u}}_t$
4: Step each environment with $\boldsymbol{u}_t^*$ with probability β, otherwise with $\hat{\boldsymbol{u}}_t$
5: Append $\boldsymbol{u}_t^*$ to the pending windows that started at steps $t-H+1, \ldots, t$
6: Emit every completed window $(\boldsymbol{y}_{t-H+1}, [\boldsymbol{u}_{t-H+1}^*, \ldots, \boldsymbol{u}_t^*])$ and drop windows that crossed a reset
7: Update θ on the emitted windows with (3.13); update the EMA parameters
8: **end for**

With a one-step horizon the constraint on β disappears, and the intervention schedule becomes a design choice. The code base implements three schedules for the recurrent baseline and the one-step case: a fixed β, a linear decay from β_{start} to β_{end} over a given number of iterations, and a SafeDAgger rule that hands control to the teacher when the RMSE between the deterministic teacher and student commands exceeds a takeover threshold of 0.10 and returns it when the error falls below a release threshold of 0.075; the hysteresis produces coherent corrective segments rather than per-step flicker. Which schedule allows a cyclic student to learn both the turn and the transitions is an experimental question that Chapter 4 answers.

3.5.6 Baseline Students

Three deterministic students serve as baselines under the same observation and action contracts. The behavior-cloning baseline consumes an eight-frame history of measured joint positions and velocities and predicts two future commands with a 512–512–512 MLP with batch normalization; it acts at the physics rate and follows a reference cursor, which makes it a plumbing and local-imitation diagnostic rather than a candidate for deployment. The recurrent DAgger baseline follows the DextrAH distillation recipe: a 512-unit layer-normalized LSTM followed by a 512–512–256 ELU MLP and a Gaussian head, trained by deterministic action regression on the teacher command, with the student's deterministic mean as the rollout command. The Transformer regression baseline is a deterministic variant of ACT [**Zhao2023**] without the variational objective: four layers, eight heads, 512-D hidden width, 2048-D feed-forward layers, and eight learned action queries that predict an eight-step chunk. The last two baselines isolate the contribution of the generative head from that of chunking and of the backbone.

3.6 Evaluation Protocol

3.6.1 Evaluation Ladder

A failed closed-loop rollout can originate in the supervised fit, in the command conversion and controller, or in the distribution shift between teacher and student states. The three evaluation modes in Figure 3.5 separate these causes, and each mode makes a stronger claim than the previous one.

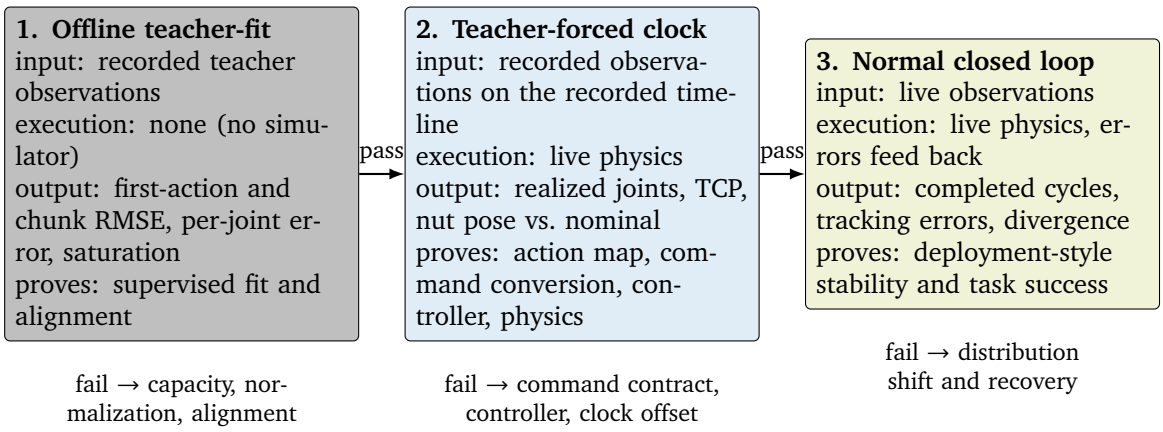


Figure 3.5: Evaluation ladder. Offline teacher-fit feeds recorded teacher observations to the student and compares predicted chunks with recorded commands without a simulator. Teacher-forced clock executes the student's commands in live physics while the student receives the recorded teacher observations on the recorded timeline, removing observation drift. Normal closed loop feeds live observations back to the student and is the only mode that measures compounding error.

In the offline teacher-fit mode, every recorded teacher observation of a demonstration is fed to the student, and the sampled chunk is compared with the recorded command chunk; the reported quantities are the RMSE of the first action and of the full chunk, per-joint errors over time, and target saturation. Passing shows that the checkpoint can represent the observation-to-action mapping under the chosen alignment; it establishes neither controller correctness nor closed-loop success. In the teacher-forced clock mode, the student's pre-

dicted commands are executed in live physics, but the student receives the recorded teacher observations selected on the recorded playback timeline, and for progress-conditioned checkpoints the same timeline supplies p_t; the realized joints, tool center point, and nut pose are compared with the recording. Passing shows that the learned action map, the command conversion, the controller, and the physics reproduce the nominal behavior once closed-loop observation drift is removed. Only the normal closed-loop mode, in which the student consumes live observations and its errors feed back into later observations, measures task success and compounding error; it is the mode of all results that claim success. The two diagnostic modes are used in that order to localize a failure before expensive retraining: a bad offline fit points to capacity, normalization, or alignment; a good fit with bad teacher-forced execution points to the command contract; and good teacher-forced execution with a bad closed loop points to distribution shift and recovery, which is the regime that motivates online labels or disturbance augmentation.

3.6.2 Metrics

Each condition is evaluated over $N_{\mathrm{tr}} = 10$ trials with up to ten cycles per trial, where each cycle consists of a 60° clockwise turn followed by release and return. Three quantities are reported. The mean completed cycle count $\bar{C} = N_{\mathrm{tr}}^{-1} \sum_i C_i$ counts, for trial i, the cycles $C_i \in \{0, \ldots, 10\}$ completed before failure or timeout; incomplete cycles are excluded, and higher is better. The cyclic tracking error is computed for every completed cycle k of trial i as $e_{i,k} = \Delta\varphi_{i,k} - 60°$, where $\Delta\varphi_{i,k} = \varphi_{i,k} - \varphi_{i,k-1}$ is the clockwise nut rotation; over all $M = \sum_i C_i$ completed cycles the mean $\mu_{\mathrm{cyc}} = M^{-1} \sum_{i,k} e_{i,k}$ and standard deviation σ_{cyc} are reported, where negative values indicate lag and positive values overshoot, and a mean closer to zero with a smaller deviation is better; if $M = 0$ both statistics are undefined. The ten-cycle tracking error is computed over the set $I_{10} = \{i : C_i = 10\}$ of trials that complete all cycles as $e_{10,i} = \sum_{k=1}^{10} \Delta\varphi_{i,k} - 600°$, reported as mean μ_{10} and standard deviation over I_{10}; it is undefined when no trial completes. All errors are in degrees. In simulation the nut angle is read from the simulator; on hardware it is read from an angular scale mounted under the bolt (Chapter 5).

Chapter 4

Experiments in Simulation

This chapter reports the simulation experiments. Section 4.1 fixes the experimental setup. Section 4.2 reports the training of the privileged teachers and the negative results on end-to-end threading that motivated the hybrid teacher. Section 4.3 reports the main student ablations in simulation, Section 4.4 the design ablations of the student, and Section 4.5 the online distillation baselines. Section 4.6 inventories every ablation that the code base supports and marks which of them have results. Numbers that are not yet available are marked TBD and listed in the draft to-do file.

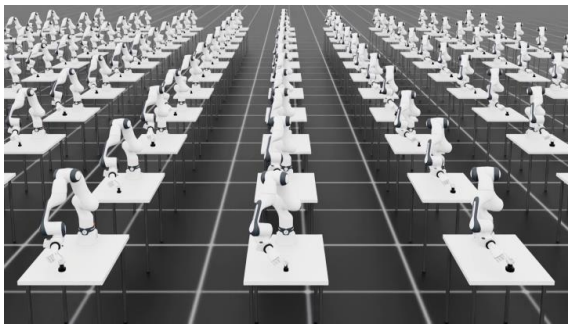
4.1 Experimental Setup

All simulation experiments use the system of Section 3.2 in Isaac Sim 5.1 with physics at 120 Hz and control at 15 Hz, on a single NVIDIA GeForce RTX 5090. Teachers are trained with 512 parallel environments for 200 epochs, which takes about ten hours per teacher; the student is trained offline for 300 epochs in about 30 minutes. The student data set is the one of Section 3.4.5: 20 successful ten-cycle demonstrations of the M24 hexagonal nut collected from 32 environments with 30 random arm impulses each, 19 486 frames in total, under lighting and texture randomization of the dome light, the tabletop, the nut and bolt textures, the floor texture, and the image background in the style of DextrAH-RGB [Sin+25]. Figure 4.1 shows the parallel training scenes and the randomized student views. Unless stated otherwise, students use the Cartesian action contract with $H = 8$, sixteen Euler steps, a temporal-ensemble decay of 0.5, EMA weights, and the DP3-style encoder; evaluation follows Section 3.6 with ten trials of ten cycles on the M24 nut in a nominal scene without physical domain randomization.

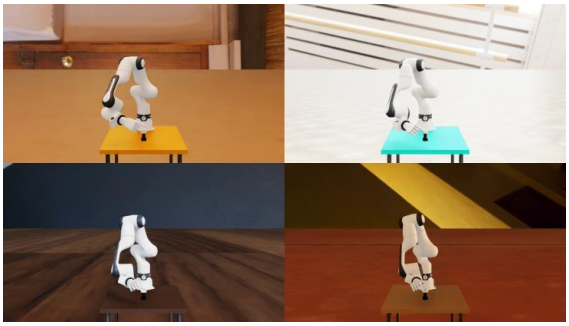
4.2 Teacher Training

4.2.1 End-to-End Threading Does Not Train

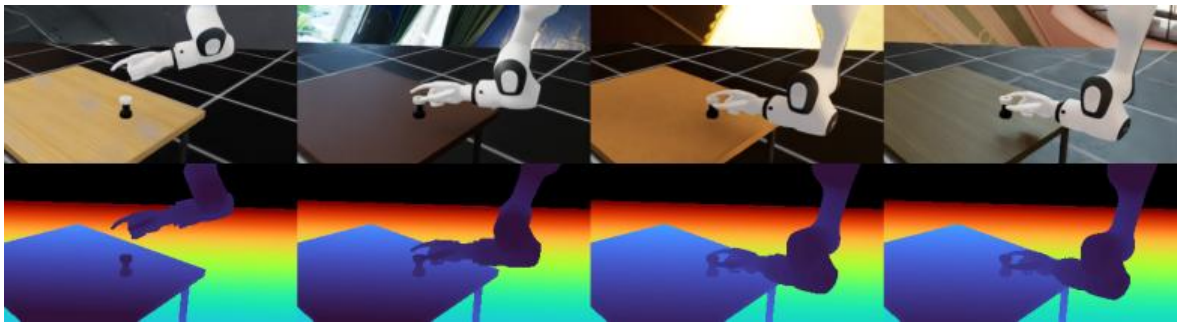
The first design of this project, laid out in the exposé, trained a single PPO policy to acquire a pinch grasp and thread a nut on faithfully simulated SDF threads. Over more than forty reward and controller variants on two robot platforms, this did not yield a usable threading policy, and the pattern of failures is informative. With the nut pre-seated in the pinch and the thread modeled by SDF colliders, a policy reached about 85 % of a geometric success criterion, but replay showed that the criterion was satisfied by pressing the nut axially through



(a) Parallel teacher-training environments.



(b) Lighting and texture randomization.



(c) Student RGB (top) and depth (bottom) observations under visual randomization.

Figure 4.1: Simulation scenes. Teacher policies are trained in hundreds of parallel workcells; demonstrations for the student are rendered with randomized dome lighting, table, object, floor, and background textures, from which the student sees aligned RGB and depth. Images reproduced from the accompanying paper [**NUTS2026**].

the thread body rather than by turning it; every attempted remedy, including lower depenetration velocities, alignment-gated descent rewards, contact-force penalties, and cumulative-advance gates, either reopened the exploit or removed the descent gradient entirely. With the nut spawned free on the bolt, the pinch itself became the obstacle: the policy either batted the nut off its spawn or hovered in an aligned pose without closing, because the shaped approach reward paid out before any contact was established. With helical waypoint tracking rewards ported from SimToolReal [Ked+26], grasps survived but the nut never wound onto the thread. The conclusion drawn from these experiments is twofold. A reward that simultaneously shapes grasp acquisition, grasp retention, helical progress, and clean termination over a multi-cycle horizon could not be found by iteration, and the SDF thread contact offered the policy a cheaper solution than threading. Both problems are removed by the design of Chapter 3: the simplified thread model makes axial progress a kinematic consequence of rotation, and the hybrid teacher assigns the transition phases, which are easy to script but hard to reward, to scripted controllers.

4.2.2 Threading and Pickup Experts

With the simplified thread model and the three-stage reward of Section 3.3, the threading expert learns the single 60° turn from the engaged nut. On the Tiangong 2 platform the recipe with the ±30° initial-yaw curriculum reached a population success rate of 83.6 % at the end of training with a peak of 90.0 %; on the FR3 platform the retained M24 teacher, trained for 200 epochs with the curriculum and the axial-height randomization, completed ten consecutive cycles in all 32 hybrid-teacher candidate environments without a single transition failure, which is the property that matters for demonstration generation. Table 4.1 summarizes the

teacher results; the single-turn success rate of the FR3 teacher at the final curriculum level is TBD.

Table 4.1: Teacher training results. Success is the population success rate at the end of training; the hybrid-teacher column counts candidate environments that completed the required cycles with zero transition failures.

Teacher	Platform	Success	Peak success	Hybrid rollout
Threading, ±30° yaw curriculum	Tiangong 2	83.6 %	90.0 %	–
Threading, M24, yaw and depth randomization	FR3	TBD	TBD	32/32 ten-cycle
Threading, M24/M30/M36 one-hot	FR3	TBD	TBD	TBD
Pickup and engagement	Tiangong 2	0 % (G1)	–	–
Pickup and engagement	FR3	0 % (G1)	–	–

The pickup expert is the weak link of the full-task variant. On both platforms it learned to approach, pinch, and lift the loose nut, with 95.3 % grasp and 94.5 % lift success on Tiangong 2 and 99.8 % pinch success on the FR3, but it did not learn the terminal engagement: the aligned handoff above the bolt tip within 10 mm and the engagement within 5 mm held for ten steps were reached in 0 % of episodes. Reward variants that added hold-or-drop terms, velocity brakes, goal-gated drop penalties, and disturbance curricula changed the grasp and lift conversion but not the terminal success. The threading results of this thesis therefore start from an engaged nut, and nut engagement is left to future work (Chapter 7).

A teacher was also trained under the hardware-matched libfranka-style Cartesian impedance controller with the published example gains (150 N/m, 10 N m/rad, 20 N m/rad nullspace) instead of the Factory gains. Over 200 epochs the grasp-center distance fell from 180 mm to a minimum of 38 mm, but neither fingertip entered the 10 mm face tolerance, the perpendicular-pinch fraction decreased from 73 % to 22 % over training, and contact, grasp, turn, and success metrics remained zero throughout. Cartesian tracking error stayed near 25 mm under this controller, and the action-bound loss grew late in training, which indicates saturated commands. The finding is that the softer gains, not the reward, prevent the policy from reaching contact, and the reported teachers therefore keep the Factory gains in simulation while the physical controller is matched to them (Chapter 5).

4.3 Student Ablations in Simulation

The main ablation study addresses the four questions of Section 1.3: (Q1) whether a unified multi-cycle student outperforms repeated execution of a single-cycle policy, (Q2) the contribution of the point-cloud encoder, (Q3) the effectiveness of flow matching relative to deterministic behavior cloning with LSTM and Transformer backbones, and (Q4) the benefit of augmenting simulation demonstrations with human-corrected real-world trajectories. Six students are compared under matched training and evaluation budgets, changing only the component named by each ablation.

(a) *Single-cycle student.* Trained on real-world-augmented single-cycle demonstrations and invoked ten times, with its temporal context reset after each cycle.

(b) *ResNet visual encoder.* The DP3 encoder is replaced by the one-branch, ImageNet-pretrained ResNet-18 stage-2 encoder that fuses RGB, log depth, validity, and calibrated XYZ geometry into a 64-D feature; policy, objective, data, and budget are unchanged.

(c) *DP3 vision with LSTM regression.* The flow-matching Transformer is replaced by the DextrAH-RGB-style backbone, a layer-normalized 512-unit LSTM followed by a 512–

512–256 ELU MLP, trained by deterministic supervised action regression on the same data.

(d) *DP3 vision with Transformer regression.* The deterministic ACT-style Transformer of Section 3.5.6 with eight learned action queries predicts the eight-step chunk.

(e) *Simulated data only.* The multi-cycle flow student is trained on simulated hybrid-teacher demonstrations alone, excluding the human-corrected real-world data.

(f) *Full method.* One flow-matching student trained on complete multi-cycle hybrid-teacher demonstrations augmented with human-corrected real-world data.

Table 4.2 reports the simulation results on the M24 hexagonal nut. The unified multi-cycle student (f) completes all ten cycles in every trial, whereas the single-cycle student (a) completes 5.5 cycles on average; its typical failure is that it slips off the edge of the nut after a few cycles, because it learned the basic threading skill but not to follow the axial advance of the nut, which a policy that only ever saw one cycle has no reason to represent (Q1). Both deterministic regression heads fail: the LSTM backbone (c) diverges before completing a single cycle, and the Transformer backbone (d) survives slightly longer, with 1.4 cycles on average and a large positive cyclic error, before the same compounding-error problem ends the trial (Q3). Neither (c) nor (d) was advanced to hardware testing. The student trained on simulated data alone (e) performs perfectly in simulation and even reaches a smaller cyclic error and deviation than the full method, which is expected because (e) is evaluated on the distribution it was trained on, whereas (f) also absorbs real-world corrections that the simulator does not require; the value of (f) appears only on hardware (Chapter 5).

Table 4.2: Simulation ablations on the M24 hexagonal nut over ten trials. Each defined entry reports mean ± standard deviation. Completed cycles are averaged across trials, cyclic-error statistics use completed cycles only, and the ten-cycle error compares the accumulated rotation with the 600° target; it is reported as "–" when no trial completes all ten cycles. Errors are signed in degrees: negative denotes lag and positive overshoot. Results from the accompanying paper [**NUTS2026**].

Ablation	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
(a) single-cycle student	5.5 ± 0.8	-4.16 ± 3.21	–
(b) ResNet encoder	2.3 ± 0.9	$+20.17 \pm 7.80$	–
(c) DP3 + LSTM regression	0.0 ± 0.0	–	–
(d) DP3 + Transformer regression	1.4 ± 0.8	$+124.21 \pm 18.50$	–
(e) simulated data only	10.0 ± 0.0	-1.77 ± 1.02	-17.7 ± 3.23
(f) full method	10.0 ± 0.0	-2.13 ± 1.10	-21.30 ± 3.48

For Q2, the vision encoders are compared both through the closed-loop ablation (b) and through a representation probe. A shared pose decoder is trained on the frozen visual feature of each encoder to predict the nut position and orientation. The DP3 feature yields a position error of 8 mm and a rotation error of 15.16° in simulation, whereas the ResNet feature yields a similar position error of 8.6 mm but a notably higher rotation error of 31.20°. In closed loop the ResNet student (b) completes 2.3 cycles with a large positive cyclic error, against ten for the DP3 student. The substantially larger rotational prediction error of the image-space representation suggests that it is less effective at preserving the geometry required for repeated re-grasping under viewpoint and axial change, which is exactly what the cyclic task demands.

4.4 Design Ablations of the Student

The ablations of Section 4.3 compare the final student against alternatives at the level of the paper. The design of the student itself was fixed by a sequence of controlled single-rollout ablations on six-cycle sequential-threading data, which this section reports because they document why each component is present. All runs in this section use the privileged 6-D nut-pose observation or the RGB-D observation as stated, 20 impulse-augmented demonstrations, 300 epochs, the Cartesian contract, $H = 8$, and the student-owned cyclic evaluation with a six-cycle requirement at a fixed seed; they are matched single rollouts, not success rates.

4.4.1 Action Contract

The first offline flow students used the 17-D joint-PD contract on pickup demonstrations. With privileged 6-D observations they reached a training flow loss of 0.03 and an offline teacher-fit chunk RMSE of 0.040 with correlation 0.998, but in closed loop they held a proper grip for about 1.3 s, lifted the nut by up to 10.5 cm, and then released it before the handoff goal; the best handoff distance was 36 mm against a 10 mm tolerance, and no trial succeeded. Terminal-hold padding of the training windows lengthened grip retention slightly and moved the closest approach by 5 mm but did not change the outcome. A residual variant that predicts arm-position residuals instead of absolute references was numerically stable but did not grasp at all. The same demonstrations materialized in the native Cartesian contract produced, with the same student, 10/10 grasps and lifts, 5/10 terminal successes on the handoff-only task, and 8/10 qualified handoffs, 7/10 engagement entries, and 2/10 held engagements on the stricter engagement task. On the full-task demonstrations with the joint-PD contract, both a privileged and an RGB-D student that reproduced their labels offline diverged within the first half second of closed-loop control and achieved 0 % pickup, with controlled-joint RMSE of 14° and 27°, respectively, against the nominal trajectory. These results, together with the zero-order-hold argument of Section 3.2.3, fixed the Cartesian contract for all later students. Table 4.3 summarizes them.

Table 4.3: Action-contract ablation of the offline flow student on pickup demonstrations (privileged 6-D observation, single seed). G0 is the handoff waypoint above the bolt tip and G1 the engagement goal.

Contract	Task	Grasp	Lift	G0	G1 held
17-D joint-PD, absolute	engagement	10/10	10/10	0/10	0/10
17-D joint-PD, terminal padding	engagement	10/10	10/10	0/10	0/10
10-D joint-PD, residual	engagement	0/10	0/10	0/10	0/10
9-D Cartesian (native)	handoff only	10/10	10/10	5/10	–
9-D Cartesian (native)	engagement	10/10	10/10	8/10	2/10

4.4.2 Progress and Phase Conditioning

On six-cycle sequential-threading demonstrations with policy-only impulses, a state-conditioned flow student completed four cycles and failed the physical return of the fifth with a 141 mm position residual, although its offline fit was excellent (first-action RMSE 0.005). With impulses in every phase segment and the global trajectory progress p_t added to the condition,

the student completed all six required cycles in 735 control ticks with zero transition failures; its release times, however, drifted increasingly behind the teacher, by +8.6 s at the sixth release, and the fifth cycle took twice as long as the others. Adding the three-way process-phase encoding reduced the completion time to 609 ticks, equalized the cycle durations to 6.7 to 7.1 s, reduced the release-time errors against the teacher to within ±0.5 s, reduced the controlled-joint RMSE against the nominal trajectory from 11.5° to 4.7° and the TCP position RMSE from 75 mm to 28 mm, and reduced the final cumulative nut-turn error from −60° to −8°. The offline teacher-fit of the phase-conditioned checkpoint was marginally worse than that of the progress-only checkpoint (first-action RMSE 0.0046 against 0.0044), so the closed-loop improvement is not explained by a better fit; the phase encoding acts as a synchronization signal that keeps the student's internal clock aligned with physical events when the cycle duration deviates from nominal. Both conditions have been enabled by default since. Table 4.4 collects the numbers.

Table 4.4: Conditioning ablation on six-cycle sequential threading (privileged 6-D observation, matched single rollouts at one seed). RMSE values are against the nominal teacher trajectory in absolute time; the final turn error is the cumulative nut-rotation error after the last cycle.

Condition	Cycles	Ticks	Joint RMSE	Final turn error
state only, policy-only impulses	4/6	776	14.9°	–
state + progress p_t, all-phase impulses	6/6	735	11.5°	−60.3°
state + progress + phase $\boldsymbol{e}_t^m$	6/6	609	4.7°	−8.2°

4.4.3 Vision Encoder and Pretraining

The first RGB-D student used the two-branch cross-attention encoder with an auxiliary nut-position head and completed all six cycles in 617 ticks, matching the privileged student; a recollection with explicit success markers and retraining reproduced the result in 627 ticks. Initializing the DP3 encoder from a pose-regression pretraining on full-task images and fine-tuning it jointly with the flow objective did not help: the student completed three cycles and lost the grip in the fourth, and a pose-retention probe showed that joint training had moved the latent coordinate system so that the pretrained 6-D decoder no longer applied, while the jointly trained position head still achieved 0.37 mm. Pretraining the DP3 encoder on rendered sweeps of a free-moving nut from the calibrated camera reached a held-out position error of 5.9 mm and a hexagon-symmetry-aware rotation error of 46°; fine-tuning on hybrid-teacher images with phase-balanced sampling reached 1.9 mm and 3.8° on held-out teacher episodes, and 2.8 mm and 6.6° on images from a student rollout, with the largest errors during grasp occlusion in pickup. The ten-cycle DP3 student of Table 4.2 was initialized from the sweep-pretrained encoder with all encoder parameters trainable; it completed all ten cycles in 1072 ticks with a maximum turn of 60.3°, a mean release-time lag of 2.7 s against its source demonstration that reached 7.4 s by the tenth cycle, and a mean absolute retained-turn error of 3.1°, which indicates timing drift rather than lifecycle failure. A closed-loop comparison of the DP3, Adapt3R-style, and ResNet encoders under identical data and budget beyond ablation (b) is TBD (Table 4.5).

Table 4.5: Vision-encoder ablation of the flow student on the ten-cycle M24 task (ten trials). The DP3 row is the full method; the remaining rows are placeholders.

Encoder	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
DP3 point cloud (RGB + XYZ)	10.0 ± 0.0	-2.13 ± 1.10	-21.30 ± 3.48
ResNet-18 early fusion (b)	2.3 ± 0.9	$+20.17 \pm 7.80$	–
Adapt3R-style (frozen ResNet + 3D attention)	TBD	TBD	TBD
Two-branch cross-attention RGB-D	TBD	TBD	TBD
Depth-only CNN	TBD	TBD	TBD

4.4.4 Privileged Observations, Horizon, Integration Steps, and Inputs

The privileged observation modes serve as upper bounds and as diagnostics: with the nut position (3-D) or pose (6-D) appended to the proprioception instead of the visual feature, the same flow student is trained and evaluated without rendering. On six-cycle sequential threading the privileged 6-D student and the RGB-D student both completed six cycles at matched settings, which shows that the visual feature carries the information the task needs. The remaining sweeps over the action horizon H, the number of Euler steps N, the temporal-ensemble decay, the hand-velocity input, and the terminal-hold padding are implemented as command-line options of the offline trainer and evaluator but have not been run as controlled studies; Table 4.6 gives the skeleton.

Table 4.6: Student hyperparameter sweeps on the ten-cycle M24 task (ten trials). All cells are placeholders except the reference row.

Sweep	Value	Completed cycles ↑	Cyclic error → 0	Latency
reference	$H = 8$, $N = 16$, decay 0.5	10.0 ± 0.0	-2.13 ± 1.10	TBD
horizon H	1 / 4 / 16	TBD	TBD	TBD
Euler steps N	1 / 4 / 8 / 32	TBD	TBD	TBD
ensemble decay	0 / 0.8	TBD	TBD	–
hand velocities omitted	26-D proprioception	TBD	TBD	–
privileged 3-D / 6-D nut pose	no vision	TBD	TBD	–

4.4.5 Nut-Size Conditioning, Domain Randomization, and Force Observations

Three further ablations are implemented but not yet qualified in simulation. The multi-size teacher conditions on a one-hot code of M24, M30, and M36 with round-robin assignment and per-size meshes, keypoints, and pitches; the student is trained on hexagonal nuts of all three sizes without an explicit size input and must infer the size from the point cloud. Per-size simulation results of the student are TBD; the hardware results across sizes are reported in Chapter 5. Physical domain randomization of mass, friction, controller gains, and reset pose is enabled during teacher training and disabled during demonstration collection and evaluation; its effect on the student when enabled at evaluation time is TBD. Hardware-equivalent force observations, the thumb and index flexion forces projected onto the pad normals and the seven arm joint reaction torques normalized by the FR3 limits, are implemented as observation channels for both teacher and student, but are not yet part of any trained contract; their effect is TBD. Table 4.7 lists the skeleton.

Table 4.7: Robustness ablations in simulation (ten trials of ten cycles). All cells are placeholders.

Ablation	Condition	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
nut size	M24 / M30 / M36	TBD	TBD	TBD
physical DR at evaluation	off / on	TBD	TBD	TBD
force observations	none / fingertip / fingertip + torque	TBD	TBD	TBD
axial-height curriculum	20 mm / TBD	TBD	TBD	TBD

4.5 Online Distillation Baselines

Online DAgger was evaluated with the recurrent LSTM student on cyclic threading from privileged 6-D observations under the Cartesian contract, with the hybrid teacher as the label source and a ten-cycle cap per episode. Three intervention schedules were compared. With full teacher control ($\beta = 1$), the student learned the first threading turn but not the cyclic release and return motion, because it never controlled a transition and the recurrent state it accumulated under teacher control did not transfer. With full student control ($\beta = 0$) and the teacher used only for labels, the student learned the first turn and the complete release-and-return transition, but subsequent turns failed, because the student rarely reached a well-formed re-grasp from which the teacher's turn label was reachable. With a scheduled intervention that decays linearly from full teacher to full student control, the student learned the transition motion and progressed farther: its best checkpoint completed two uninterrupted student-only turn-release-return cycles and entered the third threading attempt, where it lost the proper grip and terminated. SafeDAgger intervention with the thresholds of Section 3.5.5 did not complete a single turn. The earlier full-task LSTM student with the joint-PD contract reported 0 % pickup, threading entry, and task success, with 81 % of its arm targets saturated at the best-loss checkpoint, which is the student-induced state distribution rather than a controller deficiency, since the same converted commands replay accurately. The online flow-matching student of Section 3.5.5 is implemented with the same hybrid teacher and the Cartesian contract, but no successful student rollout has been demonstrated for it; its result on the cyclic task is TBD. Table 4.8 summarizes the online baselines against the offline students.

Table 4.8: Online distillation baselines on cyclic threading (privileged 6-D observation, Cartesian contract). Cycles are student-only completed cycles of the best retained checkpoint in one evaluation rollout; the offline rows are matched single rollouts from Section 4.4.

Student	Intervention	Completed cycles	Note
online DAgger LSTM	$\beta = 1$	0	learns the turn, not the transition
online DAgger LSTM	$\beta = 0$	1	learns the transition, second turn fails
online DAgger LSTM	β decayed $1 \rightarrow 0$	2	grip lost in the third attempt
online DAgger LSTM	SafeDAgger 0.10/0.075	0	no complete turn
online DAgger flow, $H = 8$	$\beta = 1$	TBD	no qualified rollout yet
offline flow, state + progress + phase	–	6/6	Table 4.4
offline flow, RGB-D	–	10/10	Table 4.2

Table 4.9 places several students on the first rung of the evaluation ladder next to their closed-loop outcome. Every listed checkpoint reproduces its training labels with first-action RMSE below 0.04 in normalized action units and correlation above 0.99, including every checkpoint that fails in closed loop. The offline fit is therefore a necessary diagnostic, which rejected no checkpoint in this project, and not a predictor of task success; the closed-loop column is decided by the contract, the conditioning, and the data, not by the fit.

The pattern across schedules is that decreasing teacher intervention improves long-horizon state coverage and preserves transition learning, but one-step online labels of a recurrent re-

Table 4.9: Offline teacher-fit against closed-loop outcome for offline flow students (EMA weights, sixteen Euler steps, one training episode). RMSE values are in the student's action units; the joint-PD rows are in physical units, the Cartesian rows in normalized action units.

Student	Data	First-action RMSE	Chunk RMSE	Correlation	Closed loop
joint-PD, 6-D nut pose	pickup, 20 episodes	0.035	0.040	0.998	0/10 handoff
Cartesian, 6-D nut pose, state only	6-cycle threading, policy impulses	0.0050	0.0110	0.9996	4/6 cycles
Cartesian, 6-D nut pose, + progress	6-cycle, all-phase impulses	0.0044	0.0122	0.9997	6/6 cycles
Cartesian, 6-D nut pose, + progress + phase	6-cycle, all-phase impulses	0.0046	0.0129	0.9997	6/6 cycles
Cartesian, RGB-D two-branch, + aux. head	6-cycle, all-phase impulses	0.0044	0.0055	0.9997	6/6 cycles
Cartesian, DP3 from full-task pretraining	6-cycle, all-phase impulses	0.0053	0.0105	0.9995	3/6 cycles
Cartesian, DP3 from sweep pretraining	10-cycle, all-phase impulses	0.019	0.019	0.994	10/10 cycles

gression head did not produce reliable threading after repeated re-grasp cycles, whereas the offline flow student trained on impulse-augmented, progress- and phase-conditioned chunks did. The evaluation ladder localizes the difference: the online LSTM students fit their labels and execute them correctly in teacher-forced mode, and fail under their own observations, so the failure lies in recovery from the student's own deviations, which the chunked generative student handles better. Chapter 6 returns to this comparison.

4.6 Ablation Inventory

Table 4.10 lists every ablation that the code base exposes as a configuration, with the chapter that reports its result or the placeholder that awaits one. The inventory follows the three ablation axes of the student assembly, namely the vision encoder, the action head, and the observation and action contract, and the teacher, data, and evaluation options around them.

Table 4.10: Inventory of ablations supported by the code base. "Result" names the table or section with a result; TBD marks an ablation whose result is pending.

Axis	Ablation	Options	Result
student	behavior cloning (joint history MLP)	diagnostic only	no qualified checkpoint
student	online DAgger LSTM	β fixed, decayed, SafeDAgger	Table 4.8
student	offline flow matching	Transformer, MLP backbone	Table 4.2
student	online DAgger flow matching	$\beta = 1$, $H = 8$	TBD
student	regression heads	LSTM (c), Transformer (d)	Table 4.2
vision	encoder	DP3, Adapt3R, ResNet (b), two-branch, depth-only	Tables 4.2, 4.5
vision	DP3 initialization	scratch, sweep pretraining	Section 4.4.3
observation	privileged vs visual	nut XYZ, nut 6-D, RGB-D	Section 4.4.4
observation	hand velocities	included, omitted	TBD
observation	force channels	fingertip force, joint torque	TBD
observation	phase encoding	none, process phase	Table 4.4
observation	progress scalar	none, global progress	Table 4.4
action	contract	9-D Cartesian, 17-D joint-PD, residual	Table 4.3
action	horizon H, steps N, ensemble decay	sweep	TBD
teacher	intervention schedule	see student	Table 4.8
teacher	nut-size conditioning	M24, M30, M36	Chapter 5, TBD
teacher	controller backend	Factory gains, libfranka gains	Section 4.2.2
data	disturbance augmentation	none, policy-only, all phases	Table 4.4
data	visual domain randomization	on, off	TBD
data	physical domain randomization	on, off	TBD
data	human corrections	excluded (e), included (f)	Chapter 5
evaluation	ladder	offline fit, teacher-forced, closed loop	Section 4.5

Chapter 5

Experiments on Real Hardware

This chapter describes the physical system and reports the hardware experiments. Section 5.1 covers the robot, the camera and its calibration, the controller, and the measurement of the nut angle. Section 5.2 describes the replay of simulated demonstrations with human corrections and quantifies the interventions. Section 5.3 reports the closed-loop results of the distilled student on the training nut sizes, Section 5.4 on unseen nut geometries, and Section 5.5 under long-horizon stress.

5.1 Hardware Setup

The physical platform comprises the same FR3 arm and Inspire RH56DFX hand as the simulation, mounted at tabletop height in front of a bench that carries the bolt on its pedestal (Figure 5.1). The hand is attached through a 10 mm adapter flange whose transform was measured and reproduced in the simulated asset; the asset itself was regenerated from the manufacturer's URDF so that joint limits, link inertias, and the mechanical coupling of the finger joints match the physical hand. An Intel RealSense D415 mounted on a fixed stand provides the RGB-D observation at 640×480, which is center-cropped and downscaled to the 320×180 policy input as in simulation. Its pose relative to the robot base was measured by an eye-to-hand calibration with a translation fit RMSE of 27 mm and a rotation fit RMSE of 3.5° over the calibration poses; the calibrated optical pose and the intrinsics are the ones used to render the simulated camera and to back-project the point cloud, so that the student sees the same geometry in both domains.

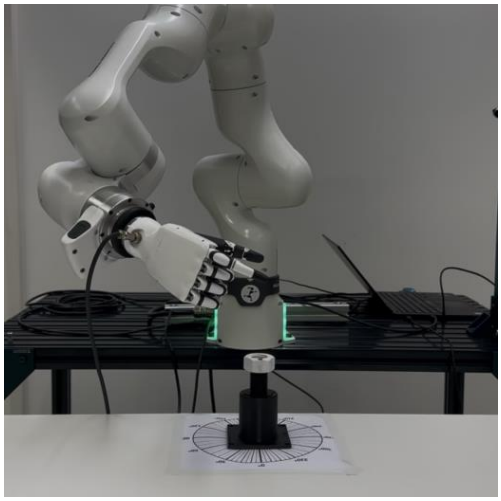
Robot commands use the same Cartesian impedance law as in simulation. The deployment controller is the libfranka Cartesian-impedance example controller, which applies $\tau = J^{\mathrm{T}}(Ke - D\dot{x})$ with a quaternion-vector orientation error, a nullspace term through a damped Jacobian pseudoinverse, and the robot's own Coriolis and gravity compensation, at the 1 kHz rate of the Franka control interface, while the student commands targets at 15 Hz. The hand is position-controlled through its serial interface. The gains of the deployment controller and the mapping of the student's bounded action to the controller target are TBD.

Controller families. The simulated Factory controller and the physical libfranka-style controller share the task-space law of (2.3) but differ in the nullspace projector. The Factory controller uses the undamped, dynamically consistent torque projector built from the joint-space inertia, whereas the libfranka example uses a damped kinematic projector built from a regularized pseudoinverse of the Jacobian transpose with regularization 0.2, and it adds the robot's Coriolis term, which the simulator supplies in the matching simulated backend. Both backends are implemented in the simulation with selectable gains, so that a teacher can

be trained under either; the teacher trained under the libfranka gains did not reach contact (Section 4.2.2), which is why the reported teachers use the Factory gains and the physical controller is tuned toward them rather than the other way around.

Trajectory export. For replay, every simulated demonstration is exported as a command-only deployment artifact. The simulator recording is authoritative for the geometric joint path; because some scripted phases change the joint position kinematically while writing zero articulation velocity, the export does not use recorded velocities but fits one twice continuously differentiable quintic curve through the recorded positions and derives position, velocity, and acceleration from it, with deployment limits on joint positions and velocities applied explicitly. Rows with a valid controller command also retain the recorded reverse-engineered 17-D impedance command, so that a joint-space replay is possible on controllers that do not expose a Cartesian interface. On the FR3 the Cartesian targets are replayed directly through the libfranka-style controller, which keeps replay and student execution on the same path.

The nut angle, which the metrics of Section 3.6.2 require, is read from an angular scale mounted under the bolt (Figure 5.2); the unwrapped angle is recorded at the end of every goal-reaching phase. The test-object family consists of M24, M30, and M36 nuts in four profiles each, hexagonal, octagonal, rectangular, and round, with matching bolts and holders; teacher and student training use only the hexagonal profiles, and the other three profiles are held out for the generalization study.



(a) Student policy threading on the physical system.



(b) Approach from the reset pose.

Figure 5.1: Real-world setup: FR3 arm with the Inspire hand, the bolt on its pedestal, and the fixed RealSense D415. Images reproduced from the accompanying paper [**NUTS2026**].



Figure 5.2: Nut-angle measurement. An angular scale under the bolt is read at the end of every goal-reaching phase to compute the cyclic and ten-cycle tracking errors; from left to right, grasp, turn, and release of one cycle. Images reproduced from the accompanying paper [**NUTS2026**].

5.2 Human-Corrected Replay

The twenty simulated ten-cycle demonstrations were replayed on the physical robot through the deployment controller. Each replay follows the recorded Cartesian targets, and the operator monitors the grasp. When slip or misalignment is anticipated, the operator takes over from grasp through release of the affected cycle, re-establishes contact, completes the turn, and hands control back to the replay. Twelve of the twenty replays, 60 %, contain at least one correction. The human-overwritten grasp-to-release segments comprise 6.48 % of the replay frames; the remaining 93.52 % retain the hybrid-teacher actions, which shows that the intervention is temporally sparse. The corrected replays are recorded with the physical camera and proprioception and form $\mathcal{D}_{\text{real}}$. The student of the full method is trained for 300 epochs on $\mathcal{D} = \mathcal{D}_{\text{sim}} \cup \mathcal{D}_{\text{real}}$ with eight-step chunks and deployed with a prediction horizon of eight and sixteen ODE integration steps.

5.3 Results on Training Nut Sizes

Table 5.1 reports the real-world counterpart of the simulation ablation on the M24 hexagonal nut. The full method (f) completes all ten cycles in every trial with a cyclic tracking error of $-4.21 \pm 1.20^\circ$ and a ten-cycle error of $-42.10 \pm 4.37^\circ$; the cyclic error is about 2° larger in magnitude than in simulation, which the thesis attributes to the thread resistance and finger compliance of the physical system that the simplified model does not capture. The single-cycle student (a) completes 5.2 cycles on average and slips off the nut edge as in simulation. The ResNet student (b) degrades further on hardware, from 2.3 to 1.1 cycles, which is consistent with an image-space representation that is more sensitive to the appearance gap than the point-cloud representation. The decisive comparison is between (e) and (f): the student trained on simulated data alone, which was perfect in simulation, completes only 2.1 cycles on hardware with a cyclic error of $-10.05 \pm 4.08^\circ$, whereas the student that also saw the human-corrected replays completes every trial. The sparse corrections, 6.5 % of the real frames, are sufficient to adapt the demonstration distribution to the contact and perception discrepancies of the hardware (Q4). The regression heads (c) and (d) were not advanced to hardware, because they did not complete a cycle in simulation.

Table 5.1: Real-world ablations on the M24 hexagonal nut over ten trials, with the same conventions as Table 4.2. Ablations (c) and (d) were not tested on hardware. Results from the accompanying paper [**NUTS2026**].

Ablation	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
(a) single-cycle student	5.2 ± 1.2	-4.56 ± 5.55	–
(b) ResNet encoder	1.1 ± 0.9	$+24.09 \pm 9.94$	–
(e) simulated data only	2.1 ± 1.4	-10.05 ± 4.08	–
(f) full method	10.0 ± 0.0	-4.21 ± 1.20	-42.10 ± 4.37

5.4 Generalization across Nut Sizes and Geometries

Table 5.2 reports the full method on the in-distribution M24, M30, and M36 hexagonal nuts and on the out-of-distribution octagonal, rectangular, and round profiles of each size, with ten trials per nut. The student completes the task on all training objects. The M30 nut achieves the smallest lag, 3.7° per cycle and 37° over the ten-cycle trial, and the M36 nut the largest among the training sizes; the thesis hypothesizes that the larger geometry increases the sensitivity to simulation-model errors such as penetration-depth mismatch, but the experiments do not isolate this factor. Among the unseen profiles, the round nut achieves the best performance, followed by the octagonal nut, in both cyclic and ten-cycle error; because no geometry factor was ablated individually, this ranking is not attributed to a specific property. The rectangular nut is a failure case. A common failure occurs after the nut has been turned 60°: in the next cycle the fingers fail to establish contact with the next pair of faces, which for a rectangle are not where the hexagon's faces would be, and the nut remains stuck at the first position. The smaller M24 rectangle escapes this failure by chance in some trials but survives only two more cycles before the mismatched re-grasp fails completely; the M30 and M36 rectangles complete no cycle. Transfer is therefore strong when an unseen profile preserves re-grasp configurations compatible with training and degrades sharply when its face topology conflicts with the learned re-grasp strategy.

Table 5.2: Geometry-dependent real-world transfer across nut sizes and profiles over ten trials per nut. Values are mean ± standard deviation; the octagonal, rectangular, and round rows are out-of-distribution geometries. Results from the accompanying paper [**NUTS2026**].

Size	Profile	Completed cycles ↑	Cyclic error → 0	Ten-cycle error → 0
M24	hexagonal	10.0 ± 0.0	-4.21 ± 1.20	-42.10 ± 4.37
	octagonal	10.0 ± 0.0	-4.10 ± 1.16	-41.00 ± 4.20
	rectangular	3.0 ± 1.2	$+30.30 \pm 4.50$	–
	round	10.0 ± 0.0	-2.33 ± 0.91	-23.30 ± 3.58
M30	hexagonal	10.0 ± 0.0	-3.70 ± 0.77	-37.00 ± 2.37
	octagonal	10.0 ± 0.0	-3.66 ± 0.58	-36.60 ± 1.74
	rectangular	0.0 ± 0.0	–	–
	round	10.0 ± 0.0	-2.45 ± 0.49	-24.50 ± 1.43
M36	hexagonal	10.0 ± 0.0	-5.52 ± 1.34	-55.20 ± 5.44
	octagonal	10.0 ± 0.0	-5.48 ± 1.22	-54.80 ± 4.75
	rectangular	0.0 ± 0.0	–	–
	round	10.0 ± 0.0	-4.78 ± 1.02	-47.80 ± 2.36

5.5 Long-Horizon Stress

Beyond the ten-cycle protocol, the student was run until failure. It eventually fails after 15 to 22 cycles, depending on nut size, geometry, and test conditions. The failures concentrate at the re-grasp: as the nut advances along the bolt by one pitch per revolution, its height after twenty cycles lies outside the axial range covered by the demonstrations, whose teacher was trained with at most 20 mm of initial-height randomization. The degradation therefore appears to be largely due to limited randomization along the bolt axis rather than to the policy's cycle count as such, and it motivates the axial curriculum proposed in Chapter 7. The per-nut cycle counts at failure are TBD.

Chapter 6

Discussion

This chapter interprets the results of Chapters 4 and 5 with respect to the four questions of Section 1.3, relates the offline and online distillation variants, discusses the action and observation contracts, returns to the questions of the exposé, and states the limitations of the work.

6.1 Interpretation of the Ablation Study

Unified multi-cycle student (Q1). The single-cycle student and the multi-cycle student share the same architecture, the same encoder, the same real-world augmentation, and the same per-cycle behavior; they differ only in whether the training data spans several cycles and in whether the temporal context is reset between cycles. The multi-cycle student completes every trial in both domains, the single-cycle student about half of the cycles. The failure mode, slipping off the nut edge after a few cycles, identifies what the single-cycle data lacks: the axial advance of the nut. In a single-cycle demonstration the nut height at grasp is always the reset height, so the student never has to read the height from the point cloud and adjust its approach; in a multi-cycle demonstration the height changes by one sixth of a pitch per cycle and the student must track it. The global progress scalar supports this, but it is not the cause of the difference, because the single-cycle student also receives it; the cause is the coverage of the axial state in the demonstrations. The result supports the central design principle: it is the unification of the complete cyclic behavior in one dataset, not a per-cycle controller invoked repeatedly, that yields a policy that sustains consecutive cycles.

Point-cloud encoder (Q2). The pose probe shows where the image-space representation falls short: its positional accuracy equals that of the point-cloud representation, but its rotational error is twice as large. Re-grasping requires the orientation of the nut faces relative to the fingers; a representation that encodes position but not orientation lets the fingers close on the right spot with the wrong face alignment, which is the error that the closed-loop ablation exhibits as a large positive cyclic error and early failure. The point-cloud encoder has no pretrained backbone and about 60 000 parameters, yet it generalizes from the randomized simulated views to the physical camera, whereas the ResNet student degrades further on hardware. Two reasons are plausible and not separated by the experiments: the point encoder consumes metric geometry that is identical in both domains up to depth noise, and random point sampling acts as a strong augmentation that the image encoder does not receive. The pretraining experiments add a caveat: initializing the encoder from a pose-regression pretraining and then training it jointly with the flow objective moved the latent so that the pretrained decoder no longer applied, and the resulting student was worse than one trained

from the sweep initialization alone. The visual feature that the flow objective shapes is not a pose estimate, and attempts to make it one through pretraining should either freeze the encoder or supervise orientation directly.

Flow matching versus deterministic regression (Q3). Both regression heads fail in simulation, the recurrent one immediately and the chunked Transformer after about one cycle, and both fail by compounding error. The chunked Transformer regression head shares the backbone, the chunk length, and the temporal ensembling of the flow student, so the difference between (d) and (f) isolates the generative objective. Two mechanisms explain the gap. The first is multimodality of the labels: the dataset contains, for near-identical observations, the teacher's nominal action, the teacher's recovery after an impulse, and the operator's correction; a squared-error head averages them and produces an action that none of the sources would take, whereas the flow head samples one of them. The second is the structure of the prediction: the flow head predicts a velocity toward the data manifold from a noisy chunk, which is a denoising problem that is easier to fit than a direct regression from observation to chunk and which, through the sixteen integration steps, refines the chunk jointly over all eight steps with full attention among them. The result does not imply that regression heads cannot thread; it implies that, at this data scale and with this heterogeneous data, the generative head is markedly more robust.

Human-corrected real-world data (Q4). The student trained on simulated data only is the best student in simulation and fails on hardware, completing two cycles with a cyclic error five times that of the full method. The 6.5 % of real frames that the operator overwrote, together with the 93.5 % of real frames that replay the teacher, are enough to close the gap. The mechanism is that the corrected segments cover exactly the states in which the simplified thread model and the real contact disagree, the grasp-to-release interval, with labels that succeed on hardware, and the uncorrected real frames supply the real camera and proprioception statistics for the rest of the cycle. The approach is a form of DAgger in which the real robot is the environment and the operator is the expert, restricted to a single round and to the segments the operator chooses. It requires no real-world reward and no real-world policy optimization, and its cost, twenty replays with sparse interventions, is small compared with the cost of real-world reinforcement learning.

6.2 Offline versus Online Distillation

The thesis set out, in the exposé and in the code base, to distill the teacher online with DAgger, and it ends with an offline flow student. The comparison in Section 4.5 explains why. The online recurrent students learn either the turn or the transition depending on who controls the environment, and a decayed intervention schedule lets them learn both but not sustain them beyond two cycles. The evaluation ladder shows that these students fit their labels and execute them correctly under teacher-forced observations, so their failure is one of recovery under their own observations. The offline flow student recovers better because its training data contain explicit recoveries from disturbances, because its chunked generative prediction with temporal ensembling smooths single-step errors, and because the progress and phase conditions keep it synchronized with the physical cycle. DAgger's advantage, labels on student-visited states, is obtained here in a weaker form by the impulses, which perturb the teacher rather than the student; the online flow variant with $\beta = 1$ obtains labels on teacher-visited states only and therefore reduces to online collection of the same

data, and the chunked-label constraint prevents the mixture schedules that help the one-step recurrent student. The online variant with student-visited states would need multi-step teacher recovery fragments collected from student states, which the code base plans but does not yet implement. The practical argument is decisive for deployment: the real-world data is offline by nature, so an offline student is required in any case, and a student that trains from the mixed dataset without changing its objective is preferable to one that needs an online loop in simulation and a separate fine-tuning stage for real data.

6.3 Action and Observation Contracts

The project implemented two action contracts and settled on the Cartesian one. The joint-PD contract was motivated by deployment convenience and by the expectation that joint references are easier for a controller to track than Cartesian targets, which they are: recorded joint references replayed at the physics rate reproduce the teacher motion with sub-millimeter accuracy. The contract fails as a learning target, not as a control interface. Converting the teacher's bounded Cartesian action into seventeen physical-unit references at the physics rate and then asking the student to emit one such reference per 15 Hz step introduces a hold error that the teacher never produced, mixes units with scale differences of three orders of magnitude in one regression target, and removes the bolt-relative structure of the action that lets the student generalize across nut heights. Every offline student trained under this contract reproduced its labels and diverged in closed loop, in the full-task case within half a second. The Cartesian contract keeps the student in the teacher's own action space and executes it through the unchanged controller, including the action smoothing that the teacher was trained with. The thesis does not conclude that joint-space students are infeasible; it concludes that a student should imitate the action the teacher produced, in the units the teacher produced it, and let the controller do the conversion it was designed for.

The observation contract follows the same logic. The student receives the previous filtered action and the controlled joints, which is what the teacher's history contains, and the two low-dimensional conditions that the demonstration provides without privileged state. The global progress scalar is the weakest part of this contract: it assumes a nominally timed trial, and the long-horizon stress trials show that the student's internal timing drifts behind the teacher by several seconds after ten cycles. The process-phase encoding corrects this within a cycle, but a trial that runs much longer than the demonstrations clamps the progress at one and leaves the student without a clock. Inferring the phase from the observation history inside the policy, rather than from an external signal, is the natural next step and is discussed in Chapter 7.

6.4 Answers to the Questions of the Exposé

The first question of the exposé asked whether vanilla PPO can learn a force-aware threading policy with a dexterous hand–arm system in a privileged, high-fidelity simulation without constraining the thread interaction. The answer from this work is negative in the form asked: without a constraint on the thread interaction, PPO exploited the SDF contact instead of threading, and with grasp acquisition, grasp retention, helical progress, and termination all shaped in one reward, no reward variant produced a threading policy. The answer is positive once the thread is constrained kinematically and the task is reduced to one turn from an engaged nut: PPO then learns a robust turn under initial-pose and controller randomization,

and the hybrid teacher extends it to arbitrarily many cycles. Force awareness, the second part of the question, was not needed for the turn: the actor of the retained teacher observes no force channel, and hardware-equivalent force observations were implemented only at the end of the project and remain untested.

The second question asked whether the privileged teacher can be distilled into a vision-based student with the localization accuracy that sub-millimeter thread interaction requires. The answer is that the student does not need to localize the nut to sub-millimeter accuracy, because the kinematic engagement of the nut on the bolt and the compliance of the impedance controller absorb lateral error; what the student needs is the orientation of the faces for the re-grasp and the height of the nut for the approach, and the point-cloud representation provides both with a precision that completes ten cycles on hardware across three sizes. The exposé's expectation that explicit depth would be required is confirmed, and its expectation that resolution or stereo would be the deciding factor is not: the deciding factor is the geometric structure of the representation.

6.5 Limitations

The work has limitations that the results make explicit. The nut starts engaged on the bolt; autonomous engagement from a loose nut was not achieved by the pickup expert, so the full task from table to tightened nut is not demonstrated. The demonstrated behavior is a pinch-grasp re-grasp strategy tied to the face topology of a hexagon; it transfers to profiles that preserve compatible re-grasp configurations and fails on the rectangle, and the student has no mechanism to discover a different grasp. The policy fails after 15 to 22 cycles because the axial range of its demonstrations is bounded, and the global progress clock does not extend beyond the nominal trial length. The evaluation uses ten trials per condition and reports means and standard deviations without confidence intervals, and several design ablations in Chapter 4 are matched single rollouts at one seed, which establish the direction of an effect but not its variance. The teacher's thread model is a kinematic simplification, and the sim-to-real gap it creates is closed by human corrections whose collection interface and selection criteria are the operator's judgment, which is not reproducible in the strict sense. Finally, the hardware experiments were run with a measured but not deployment-validated camera calibration, and the dependence of the student on calibration error has not been quantified.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This thesis presented a framework for learning long-horizon cyclic nut threading with a dexterous hand–arm system. Rather than learning the entire task with a monolithic policy, the method decomposes demonstration generation into a reinforcement-learning turning skill, learned in a simplified thread simulation with a privileged asymmetric actor–critic, and scripted release-and-return transitions, and distills the resulting long-horizon trajectories into a unified visuomotor policy for deployment. Human corrections during real-world replay adapt the demonstration distribution to the contact and perception discrepancies that the simulation does not capture, at the cost of overwriting about six percent of the replayed frames.

The student is a conditional flow-matching policy. The thesis derived its objective from the conditional flow-matching construction, showed how it applies to action chunks conditioned on a point-cloud feature, proprioception, a global progress scalar, and a process-phase encoding, and documented the trajectory-Transformer velocity field, the Euler sampler, the parameter-space and action-space averaging, and the Cartesian action contract as implemented. Controlled ablations established the contribution of each component: the multi-cycle data, the point-cloud encoder, the generative head, the human-corrected data, the Cartesian contract, and the progress and phase conditions. Online DAgger variants with recurrent regression heads, behavior cloning, and the joint-space action contract were evaluated and found insufficient for the cyclic task, and the evaluation ladder localized their failures in recovery from the student's own deviations.

Experiments show that the distilled policy completes ten consecutive threading cycles on all training-distribution nut sizes in simulation and on hardware, with mean cyclic tracking errors below 6°, and transfers to unseen round and octagonal profiles, while rectangular profiles expose a limitation of the learned re-grasp strategy. These results suggest that structured behavior generation followed by unified policy distillation is an effective approach to long-horizon, contact-rich dexterous manipulation.

7.2 Future Work

Long-horizon stress tests reveal the first direction. The current student fails after 15 to 22 cycles because the axial drift accumulated over many cycles leaves the range covered by the demonstrations. A curriculum that progressively expands the axial variation encountered during teacher training, continuing the 20 mm initial-height randomization toward the full usable thread, would improve the long-horizon capability of the teacher and, consequently, of

the distilled student. The global progress clock should at the same time be replaced by phase inference from the observation history inside the policy, so that the student is not bound to a nominal trial duration.

The second direction is autonomous nut engagement. The task was deliberately initialized with the nut already engaged after unsuccessful attempts to learn or demonstrate engagement with reinforcement learning, teleoperation, and imitation learning; the difficulty is attributed to the narrow entry configuration that a rigid nut and the helical constraint admit. Engagement will require fine-grained fingertip-force feedback and tactile sensing; the hardware-equivalent fingertip-force and joint-torque observation channels implemented at the end of this project are the starting point, and the pickup expert's handoff and engagement goals are the open teacher problem.

The third direction concerns the distillation itself. The online flow-matching student is implemented but unqualified; collecting multi-step teacher recovery fragments from student-visited states would give it the DAgger property that the recurrent students benefited from, without violating the chunk-label constraint. The pending sweeps over the action horizon, the number of integration steps, and the temporal-ensemble decay, and the closed-loop comparison of the Adapt3R-style and point-cloud encoders, would complete the design study. Finally, the generalization study suggests extending the training-object family beyond hexagonal profiles so that the student can learn re-grasp strategies for face topologies that the hexagon does not cover.

Appendix A

Hyperparameters and Interfaces

A.1 Teacher Observations

Table A.1: Actor and critic observations of the single-size M24 threading teacher. Noise is applied to the actor only. The multi-size teacher appends a clean 3-D one-hot nut-size code to both.

Input	Component	Dim	Noise (actor)
actor (39-D)	fingertip position relative to bolt frame	3	$\mathcal{N}(0, 0.25\,\text{mm})$ per axis; bolt frame $\mathcal{N}(0, 1\,\text{mm})$ per reset
	fingertip quaternion (WXYZ)	4	random-axis angle $\mathcal{N}(0, 0.1°)$
	end-effector linear velocity	3	finite difference of noisy position
	end-effector angular velocity	3	finite difference of noisy quaternion
	fingertip position relative to held nut	3	$\mathcal{N}(0, 0.25\,\text{mm})$ per axis, each body
	held-nut quaternion (WXYZ)	4	random-axis angle $\mathcal{N}(0, 0.1°)$
	target position error	3	inherits nut noise
	target quaternion error (WXYZ)	4	inherits nut noise
	grasp-phase flag	1	none
	thumb / index flexion force	2	$\mathcal{N}(0, 1\,\text{N})$, clamped to ≥ 0
	previous filtered action	9	none
critic (69-D)	fingertip position, quaternion, velocities	13	clean
	arm joint positions	7	clean
	held-nut position, position relative to bolt, quaternion	10	clean
	bolt position and quaternion	7	clean
	controller proportional gains	6	sampled per reset
	action smoothing factor	1	sampled per reset
	thumb / index flexion force	2	clean
	position and rotation action thresholds	6	sampled per reset
	target position, quaternion, progress	8	clean
	previous filtered action	9	none

A.2 Teacher Hyperparameters

A.3 Student Hyperparameters

A.4 Dataset Summary

Table A.2: PPO hyperparameters of the privileged teacher (rl_games, asymmetric actor–critic).

Parameter	Value
parallel environments	512
epochs	200
rollout horizon	256 steps
minibatch size	512
mini-epochs per update	4
recurrent sequence length	128
discount γ / GAE τ	0.995 / 0.95
learning rate	10^{-4}, adaptive with KL threshold 0.008
clipping ϵ	0.2
critic coefficient	2
entropy coefficient	0
gradient-norm clip	1.0
bounds loss coefficient	10^{-4}
actor / critic network	LSTM 2×1024 (layer norm) $\rightarrow$ MLP 512–128–64, ELU
action distribution	diagonal Gaussian, state-dependent σ
mixed precision	yes

Table A.3: Hyperparameters of the conditional flow-matching student.

Parameter	Value
action contract	9-D Cartesian, $[-1, 1]$
action horizon H	8 (0.53 s at 15 Hz)
target offset	0 (pre-step alignment)
terminal-hold padding	yes
proprioception	29-D (10 q, 10 $\dot{q}$, previous action)
proprioception encoder	MLP 128–128, ELU
visual feature	64-D (DP3: 4096 XYZRGB points, PointNet 64–128–256, max pool)
condition width	256
progress / phase embeddings	MLP 1–256–256 / 3–256–256, ELU
flow-time embedding	sinusoidal, 64-D
velocity network	Transformer, 4 layers, 8 heads, width 512, FF 2048, GELU, pre-norm
probability path	conditional optimal transport (linear interpolant)
optimizer	Adam, learning rate 10^{-4}, batch 64, 300 epochs
parameter EMA decay	0.999
auxiliary loss	nut position from visual feature, weight 1.0
RGB augmentation	saturation/contrast/brightness $[0.5, 1.5]$, hue $[-0.15, 0.15]$, blur $p = 0.1$
sampler	Euler, $N = 16$ steps (online training: 8)
temporal-ensemble decay	0.5

Table A.4: Demonstration data used for the reported students.

Property	Value
simulated demonstrations	20 successful ten-cycle episodes from 32 candidates
frames (15 Hz)	19 486
impulses per episode	30 (one per policy, waypoint, and return segment), 0.05–0.25 N s
visual randomization	dome light, table, nut/bolt, floor textures, background
image size	320 × 180 RGB and metric depth (from 640 × 480)
DP3 points per frame	10 542 min / 12 374 median / 15 366 max valid; 4096 sampled
real-world replays	20, of which 12 with at least one human correction
human-overwritten frames	6.48 % of replay frames

Bibliography

[BK00] Bicchi, A. and Kumar, V. "Robotic grasping and contact: a review". In: *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*. IEEE, 2000, pp. 348–353. ISBN: 0-7803-5886-4. DOI: 10.1109/ROBOT.2000.844081.

[Bla+26] Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., Jakubczak, S., Jones, T., Ke, L., Levine, S., Li-Bell, A., Mothukuri, M., Nair, S., Pertsch, K., Shi, L. X., Tanner, J., Vuong, Q., Walling, A., Wang, H., and Zhilinsky, U. "π_0: A Vision-Language-Action Flow Model for General Robot Control". In: (Jan. 2026).

[CXA21] Chen, T., Xu, J., and Agrawal, P. "A System for General In-Hand Object Re-Orientation". In: (Nov. 2021).

[Che+23] Chen, Y., Wang, C., Fei-Fei, L., and Liu, C. K. "Sequential Dexterity: Chaining Dexterous Policies for Long-Horizon Manipulation". In: (Oct. 2023).

[Chi+24] Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion". In: (Mar. 2024).

[DOA15] Dietrich, A., Ott, C., and Albu-Schäffer, A. "An overview of null space projections for redundant, torque-controlled robots". In: *The International Journal of Robotics Research* 34 (11 Sept. 2015), pp. 1385–1400. ISSN: 0278-3649. DOI: 10.1177/0278364914566516.

[Fei+16] Feix, T., Romero, J., Schmiedmayer, H.-B., Dollar, A. M., and Kragic, D. "The GRASP Taxonomy of Human Grasp Types". In: *IEEE Transactions on Human-Machine Systems* 46 (1 Feb. 2016), pp. 66–77. ISSN: 2168-2305. DOI: 10.1109/THMS.2015.2470657.

[Han+24] Handa, A., Allshire, A., Makoviychuk, V., Petrenko, A., Singh, R., Liu, J., Makoviichuk, D., Wyk, K. V., Zhurkevich, A., Sundaralingam, B., Narang, Y., Lafleche, J.-F., Fox, D., and State, G. "DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality". In: (Jan. 2024). URL: https://arxiv.org/pdf/2210.13702.

[Hua+25] Huang, D., Zhang, T., Li, Y., Zhao, L., Li, J., Fang, Z., Xia, C., Li, L., and He, X. "Dexterous Hand Manipulation via Efficient Imitation-Bootstrapped Online Reinforcement Learning". In: (Mar. 2025).

[Ked+26] Kedia, K., Lum, T. G. W., Bohg, J., and Liu, C. K. "SimToolReal: An Object-Centric Policy for Zero-Shot Dexterous Tool Manipulation". In: (Feb. 2026).

[KBP13] Kober, J., Bagnell, J. A., and Peters, J. "Reinforcement learning in robotics: A survey". In: *The International Journal of Robotics Research* 32 (11 Sept. 2013), pp. 1238–1274. ISSN: 0278-3649. DOI: 10.1177/0278364913495721.

[Lab19] Lab, E. Z. R. S. *Robot Dynamics Lecture Notes*. 2019.

[Lip+23] Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. "Flow Matching for Generative Modeling". In: (Feb. 2023).

[Lip+24] Lipman, Y., Havasi, M., Holderrieth, P., Shaul, N., Le, M., Karrer, B., Chen, R. T. Q., Lopez-Paz, D., Ben-Hamu, H., and Gat, I. "Flow Matching Guide and Code". In: (Dec. 2024).

[Lum+24] Lum, T. G. W., Matak, M., Makoviychuk, V., Handa, A., Allshire, A., Hermans, T., Ratliff, N. D., and Wyk, K. V. "DextrAH-G: Pixels-to-Action Dexterous Arm-Hand Grasping with Geometric Fabrics". In: (Oct. 2024).

[Luo+25] Luo, J., Xu, C., Wu, J., and Levine, S. "Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning". In: *Science Robotics* 10 (105 Aug. 2025). ISSN: 2470-9476. DOI: 10.1126/scirobotics.ads5033.

[Man+23] Manyar, O. M., Narayan, S. V., Lengade, R., and Gupta, S. K. "Physics-Informed Learning to Enable Robotic Screw-Driving Under Hole Pose Uncertainties". In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2023, pp. 2993–3000. ISBN: 978-1-6654-9190-7. DOI: 10.1109/IROS55552.2023.10342151.

[Nar+22] Narang, Y., Storey, K., Akinola, I., Macklin, M., Reist, P., Wawrzyniak, L., Guo, Y., Moravanszky, A., State, G., Lu, M., Handa, A., and Fox, D. "Factory: Fast Contact for Robotic Assembly". In: (May 2022).

[Nos+25] Noseworthy, M., Tang, B., Wen, B., Handa, A., Kessens, C., Roy, N., Fox, D., Ramos, F., Narang, Y., and Akinola, I. "FORGE: Force-Guided Exploration for Robust Contact-Rich Manipulation under Uncertainty". In: (Jan. 2025).

[Oer26] Oertel, C. "DAgger-Based Distillation of an RL-Trained Privileged Grasping Policy for a Dexterous Hand–Arm System: Towards Safer Training". Semester thesis. Technical University of Munich, Apr. 2026.

[Pet+23] Petrenko, A., Allshire, A., State, G., Handa, A., and Makoviychuk, V. "DexPBT: Scaling up Dexterous Manipulation for Hand-Arm Systems with Population Based Training". In: (May 2023).

[RGB11] Ross, S., Gordon, G. J., and Bagnell, J. A. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning". In: (Mar. 2011).

[Sin+25] Singh, R., Allshire, A., Handa, A., Ratliff, N., and Wyk, K. V. "DextrAH-RGB: Visuomotor Policies to Grasp Anything with Dexterous Hands". In: (Feb. 2025).

[Tan+23] Tang, B., Lin, M. A., Akinola, I., Handa, A., Sukhatme, G. S., Ramos, F., Fox, D., and Narang, Y. "IndustReal: Transferring Contact-Rich Assembly Tasks from Simulation to Reality". In: (May 2023).

[TJ23] Tang, L. and Jia, Y.-B. "Robotic Fastening with a Manual Screwdriver". In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2023, pp. 5269–5275. ISBN: 979-8-3503-2365-8. DOI: 10.1109/ICRA48891.2023.10161139.

[TJX24] Tang, L., Jia, Y.-B., and Xue, Y. "Robotic Manipulation of Hand Tools: The Case of Screwdriving". In: *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, May 2024, pp. 13883–13890. ISBN: 979-8-3503-8457-4. DOI: 10.1109/ICRA57147.2024.10610831.

[Wei+24] Weinberg, A. I., Shirizly, A., Azulay, O., and Sintov, A. "Survey of learning-based approaches for robotic in-hand manipulation." In: *Frontiers in robotics and AI* 11 (2024), p. 1455431. ISSN: 2296-9144. DOI: 10.3389/frobt.2024.1455431.

[XTJ23] Xue, Y., Tang, L., and Jia, Y.-B. "Dynamic Finger Gaits via Pivoting and Adapting Contact Forces". In: *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2023, pp. 8784–8791. ISBN: 978-1-6654-9190-7. DOI: 10.1109/IROS55552.2023.10342156.

[Ze+24] Ze, Y., Zhang, G., Zhang, K., Hu, C., Wang, M., and Xu, H. "3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations". In: (Sept. 2024).

[ZC16] Zhang, J. and Cho, K. "Query-Efficient Imitation Learning for End-to-End Autonomous Driving". In: (May 2016).